

SUPPORT DE RÉVISION · L3 INFORMATIQUE

Rappel du langage C

Des bases jusqu'aux pointeurs et à `malloc`

Ce que tu vas (re)voir

Partie 1 — Les bases	Programme, types, opérateurs, `printf`/`scanf`, conditions, boucles, tableaux, chaînes, fonctions, structures.
Partie 2 — Les pointeurs	Adresses, `&` et `*`, `NULL`, passage par adresse, pointeurs et tableaux, chaînes, structures, pointeur de pointeur.
Partie 3 — malloc & co	Pile et tas, `malloc`, `calloc`, `realloc`, `free`, chaînes et structures dynamiques, matrices, Valgrind.
Partie 4 — Bonus	Fichiers, fiche mémo, corrigés des « Teste-toi ».

Tous les programmes de ce document ont été compilés avec `gcc 13 -std=c11 -Wall -Wextra` puis exécutés. Les résultats affichés sont les sorties réelles (seules les adresses mémoire changent d'une exécution à l'autre).

Octobre 2026

Table des matières

Mode d'emploi **6**

1 Structure d'un programme et compilation **7**
1.1 Le programme minimal . . . 7
1.2 De `prog.c` à l'exécutable . . . 8

2 Variables et types **8**
2.1 Une variable, c'est quoi ? . . . 8
2.2 Les types de base et leur taille . . . 8
2.3 Déclarer et initialiser : les différentes possibilités . . . 9
2.4 Les constantes : trois façons de faire . . . 9
2.5 Les limites des types : dépassement et précision . . . 10
2.6 Les conversions de type . . . 11

3 Les opérateurs **11**
3.1 Arithmétiques . . . 11
3.2 Affectation et raccourcis . . . 12
3.3 Incrémentation : `i++` ou `++i` ? . . . 12
3.4 Comparaisons et opérateurs logiques . . . 12
3.5 Le piège n° 1 du débutant : = au lieu de == . . . 13
3.6 L'opérateur ternaire . . . 13
3.7 Priorité des opérateurs (du plus fort au plus faible) . . . 13
3.8 Les opérateurs bit à bit (aperçu) . . . 14

4 Entrées et sorties : `printf` et `scanf` **14**
4.1 `printf` : les formats et leurs variantes . . . 14
4.2 `scanf` : lire au clavier . . . 15
4.3 Le piège du caractère `'\n'` qui reste dans le tampon . . . 15
4.4 Vérifier que la saisie est valide . . . 16

5 Les conditions **17**
5.1 `if`, `else if`, `else` . . . 17
5.2 `switch` : choisir parmi des valeurs précises . . . 17

6 Les boucles **19**
6.1 `for` : quand on connaît le nombre de tours . . . 19
6.2 `while` : tant que la condition est vraie . . . 20
6.3 `do ... while` : au moins une fois . . . 20
6.4 `break` et `continue` . . . 20
6.5 Boucles imbriquées . . . 21

7 Les tableaux **21**
7.1 Principe . . . 21
7.2 Déclarer et initialiser : toutes les possibilités . . . 21
7.3 Parcourir : somme, moyenne, maximum . . . 22
7.4 On ne copie pas un tableau avec = . . . 23
7.5 Tableaux à deux dimensions . . . 23

8 Les chaînes de caractères **24**
8.1 Un tableau de `char` terminé par `'\0'` . . . 24
8.2 `sizeof` ou `strlen` ? . . . 24
8.3 Parcourir une chaîne . . . 24
8.4 Les fonctions de `<string.h>` . . . 25
8.5 Comparer deux chaînes : jamais avec == . . . 25
8.6 Lire une chaîne au clavier . . . 26

9 Les fonctions **27**
9.1 Anatomie d'une fonction . . . 27

9.2 Le prototype : utiliser une fonction avant sa définition . . . 27
9.3 Le passage par valeur (et sa limite) . . . 28
9.4 Tableaux en paramètre . . . 28
9.5 La récursivité . . . 29
9.6 Portée et durée de vie des variables . . . 29

10 Les structures **30**
10.1 Regrouper des données différentes . . . 30
10.2 `typedef` : donner un nom court . . . 31
10.3 Tableau de structures et copie . . . 31

11 La mémoire et les adresses **32**
11.1 La mémoire, une longue rue de casiers . . . 32
11.2 L'opérateur `&` : « l'adresse de » . . . 32

12 Qu'est-ce qu'un pointeur ? **33**
12.1 Une variable qui contient une adresse . . . 33
12.2 Déclarer, initialiser, utiliser . . . 33
12.3 Les différentes façons d'écrire la déclaration . . . 34

13 Modifier une variable à travers un pointeur **35**
13.1 Plusieurs pointeurs, une seule variable . . . 35
13.2 Changer la cible d'un pointeur . . . 35

14 `*` et `&` : ne plus jamais les confondre **36**

15 Le type d'un pointeur et sa taille **37**
15.1 Tous les pointeurs ont la même taille . . . 37
15.2 Alors, à quoi sert le type ? . . . 37
15.3 Le pointeur générique `void *` . . . 38

16 `NULL` et les pointeurs non initialisés **38**
16.1 Le pointeur non initialisé : un papier griffonné au hasard . . . 38
16.2 `NULL` : « ne pointe vers rien » . . . 38

17 Pointeurs et fonctions : le passage par adresse **39**
17.1 Le problème : la fonction travaille sur des copies . . . 39
17.2 La solution : passer les adresses . . . 39
17.3 Renvoyer plusieurs résultats . . . 40
17.4 L'oubli du `&` dans `scanf`, enfin expliqué . . . 41
17.5 Piège mortel : renvoyer l'adresse d'une variable locale . . . 42

18 Pointeurs et tableaux **42**
18.1 Le nom d'un tableau, c'est l'adresse de sa première case . . . 42
18.2 L'arithmétique des pointeurs . . . 43
18.3 Parcourir un tableau avec un pointeur . . . 43
18.4 `*p++` ou `(*p)++` ? . . . 44
18.5 Tableau et pointeur : proches, mais pas identiques . . . 44
18.6 Ce qui se passe quand on passe un tableau à une fonction . . . 45

19 Pointeurs et chaînes de caractères **45**
19.1 `char s[]` ou `char *s` : la différence qui fait planter . . . 45
19.2 Parcourir une chaîne avec un pointeur . . . 46
19.3 Tableau de chaînes et arguments du `main` . . . 47

20 Pointeurs et structures **47**
20.1 L'opérateur flèche `->` . . . 47
20.2 Modifier une structure dans une fonction . . . 48

21 `const` et les pointeurs **49**

22 Le pointeur de pointeur **50**
22.1 Une adresse vers une adresse . . . 50
22.2 À quoi ça sert ? Modifier un pointeur dans une fonction . . . 50

23 Les erreurs fréquentes avec les pointeurs **51**

24 Pourquoi l'allocation dynamique ? **52**

25 La pile et le tas **53**

26 `malloc` pas à pas **54**
26.1 Le prototype . . . 54
26.2 La recette en 4 étapes . . . 54
26.3 Premier exemple : un seul entier . . . 54
26.4 Deuxième exemple : un tableau . . . 55
26.5 Décortiquer la ligne magique . . . 55
26.6 Les différentes façons d'écrire la taille . . . 55

27 Un tableau dont la taille est choisie à l'exécution **56**

28 `calloc` : allouer et mettre à zéro **56**

29 `realloc` : agrandir ou rétrécir un bloc **57**

30 `free` : rendre la mémoire **59**
30.1 Les règles . . . 59
30.2 Le pointeur pendant (*dangling pointer*) . . . 59

31 Chaînes de caractères dynamiques **60**
31.1 Le `+ 1` qu'on n'oublie jamais . . . 60
31.2 Lire un texte puis allouer la taille exacte . . . 60

32 Structures dynamiques **61**
32.1 Une structure allouée . . . 61
32.2 Un tableau dynamique de structures . . . 62

33 Fonctions qui allouent de la mémoire **62**
33.1 Renvoyer un bloc alloué . . . 62
33.2 Allouer via un paramètre : le retour du ** . . . 63

34 Les matrices dynamiques **64**
34.1 Méthode 1 : un tableau de pointeurs vers des lignes (`int **`) . . . 64
34.2 Méthode 2 : un seul bloc . . . 65

35 Les erreurs classiques... et Valgrind pour les trouver **66**
35.1 Valgrind, ton détecteur de fuites . . . 66
35.2 Erreur 1 : la fuite mémoire (oublier `free`) . . . 66
35.3 Erreur 2 : ne pas vérifier `NULL` . . . 67
35.4 Erreur 3 : déborder du bloc . . . 67
35.5 Erreur 4 : utiliser un bloc après `free` (*use after free*) . . . 68
35.6 Erreur 5 : libérer deux fois (*double free*) . . . 68
35.7 Erreur 6 : oublier le `+ 1` d'une chaîne . . . 69
35.8 Erreur 7 : `sizeof` sur un pointeur . . . 70
35.9 Le récapitulatif . . . 70

36 Synthèse : un tableau qui grandit tout seul **70**

37 Les fichiers (l'essentiel) **73**
37.1 Écrire puis relire un fichier texte . . . 73
37.2 Quand `fopen` échoue . . . 74

38 Fiche mémo **75**
38.1 Compiler et déboguer . . . 75
38.2 Formats `printf` / `scanf` . . . 75
38.3 Pointeurs en un coup d'œil . . . 75
38.4 Allocation dynamique en un coup d'œil . . . 76
38.5 Les 10 réflexes anti-bugs . . . 76

39 Corrigés des « Teste-toi » **76**
39.1 Partie 1 — Les bases . . . 76
39.2 Partie 2 — Les pointeurs . . . 76
39.3 Partie 3 — L'allocation dynamique . . . 77

40 Pour aller plus loin **77**
40.1 La suite logique : les listes chaînées . . . 77
40.2 Pistes de travail . . . 78
40.3 Petits défis pour t'entraîner . . . 78
40.4 Références . . . 78

Mode d'emploi

Tu as fait du C il y a un ou deux mois, et maintenant tout est un peu flou ? C'est normal : le C ne pardonne pas l'oubli, mais il revient vite quand on reprend les bons réflexes. Ce document est fait pour ça.

Comment chaque notion est présentée :

1. une explication courte, en langage simple ;
2. un **programme complet** que tu peux recopier et compiler tel quel ;
3. le **résultat réel** de son exécution, dans un encadré sombre façon terminal ;
4. les **pièges** et les **différentes façons** d'écrire la même chose.

Les encadrés que tu vas croiser :

> **À retenir**
>
> L'essentiel à mémoriser. Si tu ne lis que ça, tu as déjà 80 % du chemin.

> **Piège classique**
>
> L'erreur que presque tout le monde fait au moins une fois (souvent la veille d'un examen).

> **Analogie**
>
> Une image de la vie courante pour comprendre une notion abstraite.

> **Teste-toi**
>
> Quelques questions pour vérifier que c'est rentré. Réponds **sans regarder**, puis vérifie avec les corrigés en fin de document.

Pour compiler les exemples, utilise toujours cette commande (on l'explique juste après) :

```c
gcc -std=c11 -Wall -Wextra -g prog.c -o prog
./prog
```

Les parties 2 (pointeurs) et 3 (`malloc`) sont volontairement les plus longues : ce sont les notions qui font la différence entre « je connais la syntaxe du C » et « je comprends ce que fait mon programme ».

PARTIE 1

Les bases du C

Un rappel rapide mais complet de tout ce qu'il faut maîtriser avant d'attaquer les pointeurs : chaque notion avec un exemple qui tourne et son résultat.

1. Structure d'un programme et compilation

1.1 Le programme minimal

```c
#include <stdio.h>

int main(void)
{
    printf("Salut, le C est de retour !\n");
    return 0;
}
```

Résultat de l'exécution

```
Salut, le C est de retour !
```

Ligne par ligne :

Ligne	Rôle
`#include <stdio.h>`	Demande au préprocesseur d'inclure les déclarations de `printf`, `scanf`… (*st*andard *i*nput/*o*utput).
`int main(void)`	Le **point d'entrée** : l'exécution commence toujours ici. `int` = la fonction renvoie un entier ; `void` = elle ne prend pas d'argument.
`{ ... }`	Un **bloc** d'instructions.
`printf(...);`	Affiche du texte. `\n` = retour à la ligne. Chaque instruction finit par `;`.
`return 0;`	Renvoie 0 au système = « tout s'est bien passé ». Une autre valeur signale une erreur.

`main` peut s'écrire de deux façons :

```c
int main(void)                  /* sans arguments                          */
int main(int argc, char *argv[])   /* avec les arguments de la ligne de commande */
```

1.2 De `prog.c` à l'exécutable

Le C est un langage **compilé** : ton fichier texte est traduit en langage machine avant d'être exécuté.



Option de `gcc`	Signification
`-o prog`	Nom de l'exécutable produit (sinon il s'appelle `a.out`).
`-Wall -Wextra`	Active presque tous les **avertissements**. Indispensable : le compilateur devient ton relecteur.
`-std=c11`	Utilise la norme C11 (permet par exemple `for (int i = 0; ...)`).
`-g`	Ajoute les infos de débogage (pour `gdb` et Valgrind).
`-lm`	Lie la bibliothèque mathématique (nécessaire pour `sqrt`, `pow`... de `<math.h>`).

À retenir

Un **avertissement** (*warning*) n'empêche pas la compilation, mais signale presque toujours un vrai bug. Prends l'habitude de corriger ton code jusqu'à ce que `gcc -Wall -Wextra` n'affiche **plus rien**.

Les commentaires s'écrivent de deux façons :

```c
// commentaire sur une seule ligne
/* commentaire
   sur plusieurs lignes */
```

2. Variables et types

2.1 Une variable, c'est quoi ?

Une variable est une **boîte en mémoire** qui a :

- un **nom** (`age`) ;
- un **type** (`int`) qui fixe sa taille et la façon de lire son contenu ;
- une **valeur** (`21`) ;
- une **adresse** (l'endroit où elle est rangée en mémoire — on y revient en détail dans la partie 2).

2.2 Les types de base et leur taille

L'opérateur `sizeof` donne la taille en **octets** d'un type ou d'une variable. Elle peut varier selon la machine ; voici les valeurs sur un PC Linux 64 bits :

```c
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    printf("char   : %zu octet\n",  sizeof(char));
    printf("short  : %zu octets\n", sizeof(short));
    printf("int    : %zu octets\n", sizeof(int));
    printf("long   : %zu octets\n", sizeof(long));
    printf("float  : %zu octets\n", sizeof(float));
    printf("double : %zu octets\n", sizeof(double));
    printf("bool   : %zu octet\n",  sizeof(bool));
    return 0;
}
```

Résultat de l'exécution

```
char   : 1 octet
short  : 2 octets
int    : 4 octets
long   : 8 octets
float  : 4 octets
double : 8 octets
bool   : 1 octet
```

Type	Contient	Exemple	Format `printf`
`char`	un caractère (en fait un petit entier)	`'A'`	`%c` (ou `%d` pour son code)
`int`	un entier	`42`, `-7`	`%d`
`unsigned int`	un entier positif	`42u`	`%u`
`long`	un grand entier	`100000L`	`%ld`
`float`	un réel (≈ 7 chiffres significatifs)	`3.14f`	`%f`
`double`	un réel précis (≈ 15 chiffres)	`3.14159`	`%f` (`%lf` dans `scanf`)
`bool`	vrai / faux (`<stdbool.h>`)	`true`	`%d`

Astuce

`sizeof` renvoie une valeur de type `size_t` (un entier non signé). Son format d'affichage correct est `%zu`.

2.3 Déclarer et initialiser : les différentes possibilités

```c
int a;              /* déclaration seule : valeur INDÉTERMINÉE (n'importe quoi) */
int b = 5;          /* déclaration + initialisation                              */
int c, d, e;        /* plusieurs déclarations sur une ligne                      */
int f = 1, g = 2;   /* plusieurs initialisations                                 */
char lettre = 'P';  /* un caractère : guillemets SIMPLES                         */
double pi = 3.14159;
```

Piège classique

Une variable locale non initialisée contient une valeur **aléatoire** (ce qui traînait en mémoire). `int somme;` puis `somme += x;` donne un résultat faux. Initialise toujours : `int somme = 0;`.

Règles de nommage : lettres, chiffres et _, sans commencer par un chiffre, sans accent, sans mot-clé du C (`int`, `for`...). Le C distingue les majuscules : `age` et `Age` sont deux variables différentes.

2.4 Les constantes : trois façons de faire

```c
#include <stdio.h>

#define TAUX_TVA 0.18          /* 1. macro : remplacement de texte, sans type */

enum Jour { LUNDI = 1, MARDI, MERCREDI };   /* 3. énumération : MARDI vaut 2 */

int main(void)
{
    const double PI = 3.14159; /* 2. variable constante : typée et vérifiée */
    double prix = 10000;
```

```
    printf("Prix TTC : %.0f FCFA\n", prix * (1 + TAUX_TVA));
    printf("Aire d'un cercle de rayon 2 : %.2f\n", PI * 2 * 2);
    printf("MERCREDI vaut %d\n", MERCREDI);
    /* PI = 3; serait refusé par le compilateur */
    return 0;
}
```

Résultat de l'exécution

```
Prix TTC : 11800 FCFA
Aire d'un cercle de rayon 2 : 12.57
MERCREDI vaut 3
```

Méthode	Quand l'utiliser
`#define N 100`	Taille de tableau, valeur de configuration. Pas de type, pas de `;`.
`const double PI = 3.14;`	Valeur fixe typée : préférée pour les calculs.
`enum { ROUGE, VERT }`	Ensemble de valeurs nommées (états, jours, couleurs).

2.5 Les limites des types : dépassement et précision

Les boîtes ont une taille fixe, donc une valeur maximale. Et les réels sont stockés de façon **approchée**.

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    unsigned char octet = 255;    /* un unsigned char va de 0 à 255 */
    octet = octet + 1;            /* dépasse : on repart à 0 */
    printf("255 + 1 dans un unsigned char = %d\n", octet);

    printf("Plus grand int : %d\n", INT_MAX);

    double x = 0.1 + 0.2;
    printf("0.1 + 0.2 = %.20f\n", x);
    if (x == 0.3)
        printf("x vaut exactement 0.3\n");
    else
        printf("x ne vaut PAS exactement 0.3 !\n");
    return 0;
}
```

Résultat de l'exécution

```
255 + 1 dans un unsigned char = 0
Plus grand int : 2147483647
0.1 + 0.2 = 0.30000000000000004441
x ne vaut PAS exactement 0.3 !
```

Piège classique

Ne compare jamais deux réels avec ==. Compare plutôt leur écart à une petite tolérance : `fabs(x - 0.3) < 1e-9` (avec `<math.h>`). Et attention : dépasser `INT_MAX` avec un `int` **signé** est un *comportement indéfini* (le programme peut faire n'importe quoi), contrairement aux types `unsigned` qui « reviennent à zéro ».

2.6 Les conversions de type

```c
#include <stdio.h>

int main(void)
{
    int a = 7, b = 2;

    printf("7 / 2          = %d\n", a / b);           /* division entière */
    printf("7.0 / 2        = %.2f\n", 7.0 / 2);       /* un réel => division réelle */
    printf("(double)a / b  = %.2f\n", (double)a / b); /* conversion explicite (cast) */
    printf("(double)(a / b)= %.2f\n", (double)(a / b)); /* trop tard : 3 -> 3.00 */
    printf("(int)9.99      = %d\n", (int)9.99);       /* troncature, pas arrondi */
    return 0;
}
```

Résultat de l'exécution

```
7 / 2          = 3
7.0 / 2        = 3.50
(double)a / b  = 3.50
(double)(a / b)= 3.00
(int)9.99      = 9
```

À retenir

Si **les deux** opérandes sont des entiers, la division est **entière** (la partie décimale est jetée). Pour une moyenne : `(double)somme / n`, et non `(double)(somme / n)`.

3. Les opérateurs

3.1 Arithmétiques

```c
#include <stdio.h>

int main(void)
{
    printf("17 + 5  = %d\n", 17 + 5);
    printf("17 - 5  = %d\n", 17 - 5);
    printf("17 * 5  = %d\n", 17 * 5);
    printf("17 / 5  = %d\n", 17 / 5);     /* quotient entier */
    printf("17 %% 5  = %d\n", 17 % 5);    /* reste (modulo) ; %% affiche un % */
    printf("-17 / 5 = %d\n", -17 / 5);    /* on tronque vers zéro */
    printf("-17 %% 5 = %d\n", -17 % 5);   /* le reste a le signe du dividende */
    return 0;
}
```

Résultat de l'exécution

```
17 + 5  = 22
17 - 5  = 12
17 * 5  = 85
17 / 5  = 3
17 % 5  = 2
-17 / 5 = -3
-17 % 5 = -2
```

Le modulo `%` ne marche **que sur des entiers**. Usages typiques : tester la parité (`n % 2 == 0`), récupérer le dernier chiffre (`n % 10`), faire « boucler » un indice (`(i + 1) % taille`).

3.2 Affectation et raccourcis

```c
x = 5;      /* affectation              */
x += 3;     /* x = x + 3                */
x -= 2;     /* x = x - 2                */
x *= 4;     /* x = x * 4                */
x /= 2;     /* x = x / 2                */
x %= 3;     /* x = x % 3                */
```

3.3 Incrémentation : `i++` ou `++i` ?

Les deux ajoutent 1. La différence n'apparaît que si on **utilise la valeur** dans la même expression.

```c
#include <stdio.h>

int main(void)
{
    int x = 5;
    int y = x++;   /* post : y reçoit l'ANCIENNE valeur, puis x augmente */
    printf("Après y = x++ : x = %d, y = %d\n", x, y);

    x = 5;
    int z = ++x;   /* pré : x augmente D'ABORD, puis z reçoit la nouvelle valeur */
    printf("Après z = ++x : x = %d, z = %d\n", x, z);
    return 0;
}
```

Résultat de l'exécution

```
Après y = x++ : x = 6, y = 5
Après z = ++x : x = 6, z = 6
```

Piège classique

N'écris jamais une expression qui modifie la même variable deux fois, comme `i = i++ + ++i;` : c'est un comportement indéfini. Seul, sur sa propre ligne, `i++;` et `++i;` sont équivalents.

3.4 Comparaisons et opérateurs logiques

En C, une comparaison vaut **1 si vrai, 0 si faux**. Et dans une condition, **toute valeur non nulle est considérée comme vraie**.

Opérateur	Sens	Opérateur	Sens
==	égal	!=	différent
< >	inférieur, supérieur	<= >=	inférieur ou égal, supérieur ou égal
&&	ET logique	||	OU logique
!	NON logique		

Évaluation paresseuse : dans `A && B`, si `A` est faux, `B` n'est même pas évalué (le résultat est déjà faux). Dans `A || B`, si `A` est vrai, `B` est ignoré. On s'en sert pour se protéger :

```c
#include <stdio.h>

int main(void)
{
    int a = 10, b = 0;

    printf("5 > 3 vaut %d, 5 == 3 vaut %d\n", 5 > 3, 5 == 3);

    /* Si b vaut 0, a / b n'est JAMAIS calculé : pas de division par zéro */
```

```c
    if (b != 0 && a / b > 2)
        printf("a / b > 2\n");
    else
        printf("b vaut 0 (ou a / b <= 2) : division évitée\n");
    return 0;
}
```

Résultat de l'exécution

```
5 > 3 vaut 1, 5 == 3 vaut 0
b vaut 0 (ou a / b <= 2) : division évitée
```

3.5 Le piège n° 1 du débutant : = au lieu de ==

```c
#include <stdio.h>

int main(void)
{
    int x = 3;
    if (x = 5)          /* affecte 5 à x ! la condition vaut 5 = vrai */
        printf("x vaut 5 ? Non, x valait 3... mais maintenant x = %d\n", x);
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘main’:
prog.c:6:9: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    6 |     if (x = 5)          /* affecte 5 à x ! la condition vaut 5 = vrai */
      |         ^
```

Résultat de l'exécution

```
x vaut 5 ? Non, x valait 3... mais maintenant x = 5
```

Grâce à `-Wall`, gcc te prévient. Sans cette option, il ne dirait rien.

3.6 L'opérateur ternaire

Un `if/else` compact qui **produit une valeur** : `condition ? valeur_si_vrai : valeur_si_faux`.

```c
#include <stdio.h>

int main(void)
{
    int a = 12, b = 7;
    int max = (a > b) ? a : b;
    printf("max = %d\n", max);
    printf("%d est %s\n", b, (b % 2 == 0) ? "pair" : "impair");
    return 0;
}
```

Résultat de l'exécution

```
max = 12
7 est impair
```

3.7 Priorité des opérateurs (du plus fort au plus faible)

Niveau	Opérateurs
1	`() [] -> . x++ x--`
2	`! ++x --x *p` (contenu) `&x` (adresse) `(type) sizeof`
3	`* / %`
4	`+ -`

Niveau	Opérateurs
5	< <= > >= puis == !=
6	&& puis ||
7	? : puis = += -= ...

Astuce

Dans le doute, **mets des parenthèses**. `(a + b) * c` est plus lisible et plus sûr que de compter sur ta mémoire du tableau.

3.8 Les opérateurs bit à bit (aperçu)

Ils agissent sur chaque bit d'un entier. Très utilisés en système, réseau et sécurité (masques, drapeaux, chiffrement XOR).

```c
#include <stdio.h>

int main(void)
{
    int a = 12;   /* 1100 en binaire */
    int b = 10;   /* 1010 en binaire */
    printf("a & b  = %d  (1000 : ET)\n", a & b);
    printf("a | b  = %d (1110 : OU)\n", a | b);
    printf("a ^ b  = %d  (0110 : OU exclusif)\n", a ^ b);
    printf("1 << 3 = %d  (décalage à gauche = x 2^3)\n", 1 << 3);
    printf("a >> 2 = %d  (décalage à droite = / 2^2)\n", a >> 2);
    return 0;
}
```

Résultat de l'exécution

```
a & b  = 8  (1000 : ET)
a | b  = 14 (1110 : OU)
a ^ b  = 6  (0110 : OU exclusif)
1 << 3 = 8  (décalage à gauche = x 2^3)
a >> 2 = 3  (décalage à droite = / 2^2)
```

Ne confonds pas & (ET bit à bit) avec && (ET logique), ni | avec ||.

4. Entrées et sorties : `printf` et `scanf`

4.1 `printf` : les formats et leurs variantes

```c
#include <stdio.h>

int main(void)
{
    int n = 42;
    double d = 3.14159;

    printf("[%d]\n", n);       /* normal                       */
    printf("[%5d]\n", n);      /* largeur 5, aligné à droite   */
    printf("[%-5d]\n", n);     /* largeur 5, aligné à gauche   */
    printf("[%05d]\n", n);     /* complété avec des zéros      */
    printf("[%f]\n", d);       /* 6 décimales par défaut       */
    printf("[%.2f]\n", d);     /* 2 décimales (arrondi)        */
    printf("[%8.3f]\n", d);    /* largeur 8 dont 3 décimales   */
    printf("[%e]\n", d);       /* notation scientifique        */
    printf("[%x] [%X] [%o]\n", 255, 255, 8);  /* hexa, HEXA, octal */
    printf("[%c] [%d]\n", 'A', 'A');          /* caractère et son code ASCII */
    printf("[%s]\n", "texte");
```

```
    printf("[100%%]\n");       /* pour afficher le symbole %    */
    return 0;
}
```

Résultat de l'exécution

```
[42]
[   42]
[42   ]
[00042]
[3.141590]
[3.14]
[   3.142]
[3.141590e+00]
[ff] [FF] [10]
[A] [65]
[texte]
[100%]
```

Format	Type	Format	Type
`%d`	`int`	`%c`	`char`
`%u`	`unsigned int`	`%s`	chaîne (`char *`)
`%ld`	`long`	`%p`	adresse (pointeur)
`%f`	`double` (et `float`)	`%x`	entier en hexadécimal
`%zu`	`size_t` (résultat de `sizeof`)	`%%`	le caractère %

4.2 `scanf` : lire au clavier

`scanf` doit **écrire** dans ta variable : il a donc besoin de son **adresse**, d'où le & (tout s'éclaire dans la partie 2).

```c
#include <stdio.h>

int main(void)
{
    int age;
    double taille;

    printf("Ton âge et ta taille : ");
    scanf("%d %lf", &age, &taille);    /* %lf pour un double dans scanf ! */
    printf("Tu as %d ans et tu mesures %.2f m\n", age, taille);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Ton âge et ta taille : 21 1.78
Tu as 21 ans et tu mesures 1.78 m
```

Piège classique

Dans `scanf`, un `double` se lit avec `%lf` (alors que `printf` accepte `%f`). Et oublier le & (`scanf("%d", age);`) fait écrire n'importe où en mémoire : gcc t'avertit avec `-Wall` (voir la partie 2).

4.3 Le piège du caractère `'\n'` qui reste dans le tampon

Quand tu tapes `5` puis Entrée, le clavier envoie `5` **et** `'\n'`. `%d` lit le 5… et laisse le `'\n'`. Un `%c` qui suit le lit immédiatement, sans attendre :

```c
#include <stdio.h>

int main(void)
```

```
{
    int n;
    char c;

    printf("Un nombre : ");
    scanf("%d", &n);
    printf("Une lettre : ");
    scanf("%c", &c);                /* lit le '\n' resté en attente ! */
    printf("Code du caractère lu : %d (10 = '\\n')\n", c);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Un nombre : 5
Une lettre : Code du caractère lu : 10 (10 = '\n')
```

Le programme n'a même pas attendu la lettre. La solution : **un espace avant `%c`**, qui veut dire « saute les espaces et retours à la ligne » :

```c
#include <stdio.h>

int main(void)
{
    int n;
    char c;

    printf("Un nombre : ");
    scanf("%d", &n);
    printf("Une lettre : ");
    scanf(" %c", &c);               /* l'espace avale le '\n' */
    printf("Lettre lue : %c\n", c);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Un nombre : 5
Une lettre : A
Lettre lue : A
```

4.4 Vérifier que la saisie est valide

`scanf` renvoie le **nombre de valeurs lues avec succès**. Si l'utilisateur tape des lettres au lieu d'un nombre, il renvoie 0 :

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Entre un entier : ");
    if (scanf("%d", &n) != 1) {
        printf("Saisie invalide !\n");
        return 1;                   /* on quitte avec un code d'erreur */
    }
    printf("Tu as tapé %d\n", n);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Entre un entier : abc
Saisie invalide !
```

5. Les conditions

5.1 `if`, `else if`, `else`

```c
#include <stdio.h>

int main(void)
{
    int note;
    printf("Ta note : ");
    scanf("%d", &note);

    if (note >= 16) {
        printf("Très bien\n");
    } else if (note >= 14) {
        printf("Bien\n");
    } else if (note >= 10) {
        printf("Passable\n");
    } else {
        printf("Insuffisant\n");
    }
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Ta note : 14
Bien
```

Les conditions sont testées **dans l'ordre** : dès qu'une est vraie, son bloc s'exécute et on saute toutes les suivantes. C'est pour cela qu'on teste d'abord `>= 16`, puis `>= 14`.

Les différentes possibilités :

```c
if (x > 0) printf("positif\n");     /* sans accolades : UNE seule instruction  */

if (x > 0 && x < 100) { ... }       /* conditions combinées                    */

if (x != 0) { ... }                 /* équivalent à : if (x) { ... }           */
if (x == 0) { ... }                 /* équivalent à : if (!x) { ... }          */

if (a > 0) {                        /* conditions imbriquées                   */
    if (b > 0) { ... }
}
```

Piège classique

Sans accolades, seule la **première** instruction dépend du `if`, quelle que soit l'indentation :

```c
if (note < 10)
    printf("Échec\n");
    printf("Rattrapage\n");   /* s'exécute TOUJOURS ! */
```

Mets toujours des accolades dès qu'il y a plus d'une instruction (et idéalement, tout le temps).

5.2 `switch` : choisir parmi des valeurs précises

```c
#include <stdio.h>

int main(void)
{
    int choix;
    printf("1-Nouveau  2-Ouvrir  3-Quitter : ");
    scanf("%d", &choix);
```

```
    switch (choix) {
        case 1:
            printf("Nouveau fichier\n");
            break;              /* sort du switch */
        case 2:
            printf("Ouverture...\n");
            break;
        case 3:
            printf("Au revoir !\n");
            break;
        default:                /* aucun cas ne correspond */
            printf("Choix invalide\n");
    }
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
1-Nouveau  2-Ouvrir  3-Quitter : 3
Au revoir !
```

Possibilité : regrouper plusieurs cas. Sans instruction entre eux, plusieurs `case` partagent le même code :

```c
#include <stdio.h>

int main(void)
{
    char c = 'e';
    switch (c) {
        case 'a': case 'e': case 'i': case 'o': case 'u': case 'y':
            printf("'%c' est une voyelle\n", c);
            break;
        default:
            printf("'%c' est une consonne\n", c);
    }
    return 0;
}
```

Résultat de l'exécution

```
'e' est une voyelle
```

Piège classique

Oublier un `break` fait « tomber » l'exécution dans le cas suivant (*fall-through*). Bonne nouvelle : `-Wextra` le repère.

```c
#include <stdio.h>

int main(void)
{
    int jour = 1;
    switch (jour) {
        case 1: printf("Lundi\n");      /* pas de break... */
        case 2: printf("Mardi\n");      /* ... on continue ici */
        case 3: printf("Mercredi\n");
                break;
        default: printf("Autre\n");
    }
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘main’:
prog.c:7:17: warning: this statement may fall through [-Wimplicit-fallthrough=]
    7 |         case 1: printf("Lundi\n");      /* pas de break... */
      |                 ^~~~~~~~~~~~~~~~
prog.c:8:9: note: here
    8 |         case 2: printf("Mardi\n");      /* ... on continue ici */
      |         ^~~~
prog.c:8:17: warning: this statement may fall through [-Wimplicit-fallthrough=]
    8 |         case 2: printf("Mardi\n");      /* ... on continue ici */
      |                 ^~~~~~~~~~~~~~~~
prog.c:9:9: note: here
    9 |         case 3: printf("Mercredi\n");
      |         ^~~~
```

Résultat de l'exécution

```
Lundi
Mardi
Mercredi
```

Limites du `switch` : il ne compare qu'une valeur **entière** (ou un `char`) à des **constantes**. Pas d'intervalles (`case > 10` est impossible), pas de réels, pas de chaînes : dans ces cas, utilise `if`.

Situation	Outil adapté
Intervalles, conditions complexes, réels, chaînes	`if / else if`
Une variable entière comparée à plusieurs valeurs fixes (menu)	`switch`
Choisir entre deux **valeurs** dans une expression	ternaire `? :`

6. Les boucles

6.1 `for` : quand on connaît le nombre de tours

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 5; i++)            /* départ ; condition ; pas */
        printf("%d ", i);
    printf("\n");

    for (int i = 10; i > 0; i -= 3)         /* on peut décompter, avancer de 3... */
        printf("%d ", i);
    printf("\n");

    for (int i = 0, j = 10; i < j; i++, j--) /* deux variables à la fois */
        printf("(%d,%d) ", i, j);
    printf("\n");
    return 0;
}
```

Résultat de l'exécution

```
1 2 3 4 5
10 7 4 1
(0,10) (1,9) (2,8) (3,7) (4,6)
```

Une boucle `for (init; condition; pas)` se déroule ainsi : `init` une seule fois, puis [**test de la condition → corps → pas**] tant que la condition est vraie.

6.2 `while` : tant que la condition est vraie

```c
#include <stdio.h>

int main(void)
{
    int n = 2026, nb_chiffres = 0;
    while (n > 0) {
        n = n / 10;          /* on retire le dernier chiffre */
        nb_chiffres++;
    }
    printf("2026 a %d chiffres\n", nb_chiffres);
    return 0;
}
```

Résultat de l'exécution

```
2026 a 4 chiffres
```

6.3 `do ... while` : au moins une fois

La condition est testée **à la fin** : le corps s'exécute au moins une fois. Parfait pour **redemander une saisie** :

```c
#include <stdio.h>

int main(void)
{
    int n;
    do {
        printf("Un nombre entre 1 et 10 : ");
        scanf("%d", &n);
    } while (n < 1 || n > 10);       /* on recommence tant que c'est invalide */
    printf("Merci, tu as choisi %d\n", n);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Un nombre entre 1 et 10 : 15
Un nombre entre 1 et 10 : 0
Un nombre entre 1 et 10 : 7
Merci, tu as choisi 7
```

Boucle	Condition testée	Nombre minimal de tours	Usage typique
`for`	avant chaque tour	0	compter, parcourir un tableau
`while`	avant chaque tour	0	répéter tant qu'un état dure
`do ... while`	après chaque tour	**1**	menus, validation de saisie

6.4 `break` et `continue`

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 10; i++) {
        if (i % 3 == 0)
            continue;        /* saute la fin de CE tour, passe au suivant */
        if (i == 8)
```

```c
            break;          /* quitte complètement la boucle */
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Résultat de l'exécution

```
1 2 4 5 7
```

6.5 Boucles imbriquées

```c
#include <stdio.h>

int main(void)
{
    int n = 4;
    for (int ligne = 1; ligne <= n; ligne++) {
        for (int k = 1; k <= ligne; k++)
            printf("* ");
        printf("\n");
    }
    return 0;
}
```

Résultat de l'exécution

```
*
* *
* * *
* * * *
```

La boucle intérieure fait **tous** ses tours à chaque tour de la boucle extérieure.

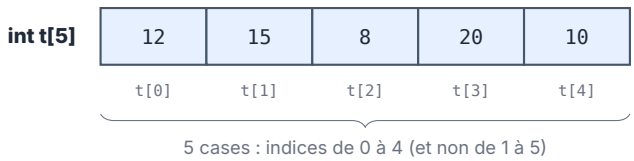
Piège classique

- **Boucle infinie** : oublier de faire évoluer la variable (`while (i < 10)` sans `i++`).
- **Décalage d'un** (*off-by-one*) : `for (i = 0; i <= 5; i++)` fait **6** tours, pas 5.
- **Point-virgule fatal** : `for (i = 0; i < 5; i++);` — le `;` est un corps vide, le bloc qui suit ne s'exécute qu'une fois.

7. Les tableaux

7.1 Principe

Un tableau est une **suite de cases du même type, collées en mémoire**, numérotées à partir de **0**.



7.2 Déclarer et initialiser : toutes les possibilités

```c
#include <stdio.h>

int main(void)
{
    int a[5] = {12, 15, 8, 20, 10};  /* initialisation complète          */
    int b[5] = {1, 2};               /* le reste est mis à 0             */
    int c[5] = {0};                  /* astuce : tout à zéro             */
```

```c
    int d[] = {4, 5, 6};            /* taille déduite : 3 cases           */
    int e[5];                       /* NON initialisé : valeurs aléatoires */
    (void)e;                        /* (évite l'avertissement "inutilisé") */

    int taille_d = sizeof(d) / sizeof(d[0]);   /* nombre d'éléments */
    printf("d contient %d éléments\n", taille_d);

    printf("b : ");
    for (int i = 0; i < 5; i++) printf("%d ", b[i]);
    printf("\nc : ");
    for (int i = 0; i < 5; i++) printf("%d ", c[i]);
    printf("\na[0] = %d, a[4] = %d\n", a[0], a[4]);
    return 0;
}
```

Résultat de l'exécution

```
d contient 3 éléments
b : 1 2 0 0 0
c : 0 0 0 0 0
a[0] = 12, a[4] = 10
```

7.3 Parcourir : somme, moyenne, maximum

```c
#include <stdio.h>

#define N 6

int main(void)
{
    int notes[N] = {12, 15, 8, 20, 10, 14};
    int somme = 0;
    int max = notes[0];          /* on part du premier élément */

    for (int i = 0; i < N; i++) {
        somme += notes[i];
        if (notes[i] > max)
            max = notes[i];
    }
    printf("Somme   : %d\n", somme);
    printf("Moyenne : %.2f\n", (double)somme / N);
    printf("Maximum : %d\n", max);
    return 0;
}
```

Résultat de l'exécution

```
Somme   : 79
Moyenne : 13.17
Maximum : 20
```

Piège classique

Le C **ne vérifie jamais** les indices. `notes[6]` ou `notes[-1]` compile et s'exécute… en lisant ou en écrivant **hors du tableau**, dans la mémoire d'autres variables. C'est un *comportement indéfini* et l'origine des fameuses failles de type *buffer overflow*. C'est à toi de garantir `0 <= i < N`.

7.4 On ne copie pas un tableau avec =

```c
int main(void)
{
    int t1[3] = {1, 2, 3};
    int t2[3];
    t2 = t1;            /* interdit ! */
    return 0;
}
```

Ce que dit le compilateur : erreur, pas d'exécutable

```
prog.c: In function ‘main’:
prog.c:5:8: error: assignment to expression with array type
    5 |     t2 = t1;            /* interdit ! */
      |        ^
prog.c:4:9: warning: variable ‘t2’ set but not used [-Wunused-but-set-variable]
    4 |     int t2[3];
      |         ^~
```

Il faut copier **case par case** avec une boucle (ou avec `memcpy` de `<string.h>`) :

```c
for (int i = 0; i < 3; i++)
    t2[i] = t1[i];
```

7.5 Tableaux à deux dimensions

```c
#include <stdio.h>

int main(void)
{
    int m[2][3] = {
        {1, 2, 3},      /* ligne 0 */
        {4, 5, 6}       /* ligne 1 */
    };

    for (int i = 0; i < 2; i++) {          /* i : numéro de ligne   */
        for (int j = 0; j < 3; j++)        /* j : numéro de colonne */
            printf("%3d", m[i][j]);
        printf("\n");
    }
    printf("m[1][2] = %d\n", m[1][2]);
    return 0;
}
```

Résultat de l'exécution

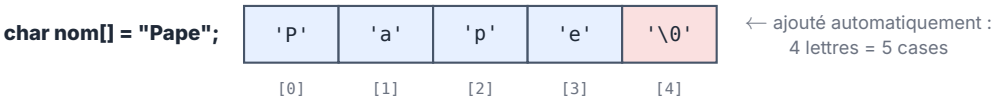
```
  1  2  3
  4  5  6
m[1][2] = 6
```

En mémoire, les lignes sont simplement rangées l'une après l'autre : `1 2 3 4 5 6`.

8. Les chaînes de caractères

8.1 Un tableau de `char` terminé par `'\0'`

Le C n'a pas de vrai type « chaîne ». Une chaîne est un tableau de `char` dont la fin est marquée par le caractère spécial `'\0'` (code 0). Toutes les fonctions de chaînes s'arrêtent sur ce marqueur.



8.2 `sizeof` ou `strlen` ?

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char a[] = "Bonjour";      /* taille déduite : 7 lettres + '\0' = 8 */
    char b[20] = "Bonjour";    /* 20 cases réservées, 8 utilisées       */

    printf("sizeof(a) = %zu, strlen(a) = %zu\n", sizeof(a), strlen(a));
    printf("sizeof(b) = %zu, strlen(b) = %zu\n", sizeof(b), strlen(b));
    return 0;
}
```

Résultat de l'exécution

```
sizeof(a) = 8, strlen(a) = 7
sizeof(b) = 20, strlen(b) = 7
```

À retenir

`sizeof` = la **place réservée** (en octets). `strlen` = le **nombre de caractères avant le `'\0'`**. Pour stocker une chaîne de n lettres, il faut $\mathbf{n + 1}$ cases.

8.3 Parcourir une chaîne

On avance jusqu'au `'\0'`, sans avoir besoin de connaître la longueur :

```c
#include <stdio.h>
#include <ctype.h>

int main(void)
{
    char s[] = "Bonjour Dakar";
    int voyelles = 0;

    for (int i = 0; s[i] != '\0'; i++) {
        char c = tolower(s[i]);
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' || c == 'y')
            voyelles++;
        s[i] = toupper(s[i]);            /* on met en majuscules au passage */
    }
    printf("%s : %d voyelles\n", s, voyelles);
    return 0;
}
```

Résultat de l'exécution

```
BONJOUR DAKAR : 5 voyelles
```

8.4 Les fonctions de <string.h>

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char prenom[30];
    char complet[60];

    strcpy(prenom, "Fatou");            /* copie : prenom = "Fatou"      */
    strcpy(complet, prenom);
    strcat(complet, " Diop");           /* concaténation (ajout à la fin) */
    printf("complet = \"%s\" (%zu lettres)\n", complet, strlen(complet));

    /* strcmp renvoie 0 si égales, <0 si la 1re est avant, >0 si après */
    printf("strcmp(\"abc\", \"abc\") = %d\n", strcmp("abc", "abc"));
    printf("\"abc\" vs \"abd\" : %s\n", strcmp("abc", "abd") < 0 ? "avant" : "après");

    /* snprintf : écrit dans une chaîne comme printf, sans déborder */
    char msg[40];
    snprintf(msg, sizeof(msg), "%s a %d ans", prenom, 22);
    printf("msg = \"%s\"\n", msg);
    return 0;
}
```

Résultat de l'exécution

```
complet = "Fatou Diop" (10 lettres)
strcmp("abc", "abc") = 0
"abc" vs "abd" : avant
msg = "Fatou a 22 ans"
```

Fonction	Rôle	Attention
`strlen(s)`	longueur sans le `'\0'`	renvoie un `size_t` (`%zu`)
`strcpy(dest, src)`	copie `src` dans `dest`	`dest` doit être assez grand
`strncpy(dest, src, n)`	copie au plus `n` caractères	n'ajoute pas toujours le `'\0'`
`strcat(dest, src)`	ajoute `src` à la fin de `dest`	`dest` doit être assez grand
`strcmp(s1, s2)`	compare (0 si égales)	**jamais** `s1 == s2`
`snprintf(dest, taille, ...)`	`printf` dans une chaîne	la plus sûre pour construire du texte

8.5 Comparer deux chaînes : jamais avec ==

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char rep[] = "oui";
    if (rep == "oui")       /* compare des ADRESSES, pas le texte */
        printf("== : égales\n");
    else
        printf("== : différentes (!)\n");

    if (strcmp(rep, "oui") == 0)       /* la bonne façon */
        printf("strcmp : égales\n");
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘main’:
prog.c:7:13: warning: comparison with string literal results in unspecified behavior
    [-Waddress]
    7 |     if (rep == "oui")       /* compare des ADRESSES, pas le texte */
      |             ^~
```

Résultat de l'exécution

```
== : différentes (!)
strcmp : égales
```

8.6 Lire une chaîne au clavier

`scanf("%s")` s'arrête au **premier espace** :

```c
#include <stdio.h>

int main(void)
{
    char nom[30];
    printf("Ton nom complet : ");
    scanf("%29s", nom);            /* 29 max : on garde 1 case pour '\0' */
    printf("Lu : \"%s\"\n", nom);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Ton nom complet : Pape Thiam
Lu : "Pape"
```

`fgets` lit **toute la ligne**, espaces compris, sans jamais déborder. Mais il garde le `'\n'` final, qu'on retire :

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char nom[30];
    printf("Ton nom complet : ");
    fgets(nom, sizeof(nom), stdin);     /* lit au plus 29 caractères */
    printf("Avant  : \"%s\"\n", nom);
    nom[strcspn(nom, "\n")] = '\0';     /* remplace le '\n' par la fin de chaîne */
    printf("Après  : \"%s\"\n", nom);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Ton nom complet : Pape Thiam
Avant  : "Pape Thiam
"
Après  : "Pape Thiam"
```

Piège classique

`strcpy`, `strcat` et `scanf("%s")` sans limite **ne vérifient pas la taille** de la destination. Copier 50 caractères dans un `char t[10]` écrase la mémoire voisine : c'est exactement le mécanisme des attaques par **débordement de tampon**. Réflexes : `scanf("%29s")`, `fgets`, `snprintf`.

9. Les fonctions

9.1 Anatomie d'une fonction

```c
type_de_retour nom(type1 param1, type2 param2)
{
    /* corps */
    return valeur;    /* du type annoncé */
}
```

```c
#include <stdio.h>

int carre(int x)
{
    return x * x;
}

double moyenne(int a, int b)
{
    return (a + b) / 2.0;
}

void afficher_ligne(int n)          /* void : ne renvoie rien */
{
    for (int i = 0; i < n; i++)
        printf("-");
    printf("\n");
}

int main(void)
{
    afficher_ligne(20);
    printf("carre(7)       = %d\n", carre(7));
    printf("moyenne(12, 15) = %.1f\n", moyenne(12, 15));
    printf("carre(carre(2)) = %d\n", carre(carre(2)));   /* imbrication */
    afficher_ligne(20);
    return 0;
}
```

Résultat de l'exécution

```
--------------------
carre(7)       = 49
moyenne(12, 15) = 13.5
carre(carre(2)) = 16
--------------------
```

9.2 Le prototype : utiliser une fonction avant sa définition

Le compilateur lit le fichier **de haut en bas**. Si `main` appelle une fonction écrite plus bas, il faut l'annoncer avec un **prototype** (la première ligne, suivie d'un `;`) :

```c
#include <stdio.h>

int maximum(int a, int b);          /* prototype : "cette fonction existe" */

int main(void)
{
    printf("maximum(4, 9) = %d\n", maximum(4, 9));
    return 0;
}

int maximum(int a, int b)           /* définition, plus bas */
{
    return (a > b) ? a : b;
```

```
}
```

Résultat de l'exécution

```
maximum(4, 9) = 9
```

Trois organisations possibles : (1) écrire les fonctions **avant** `main` ; (2) mettre les **prototypes en haut** et les définitions après `main` ; (3) pour les gros projets, mettre les prototypes dans un fichier `.h` inclus avec `#include "mon_fichier.h"`.

9.3 Le passage par valeur (et sa limite)

Quand tu appelles une fonction, elle reçoit une **copie** des valeurs. Modifier le paramètre ne change **pas** la variable d'origine :

```c
#include <stdio.h>

void tenter_de_doubler(int x)
{
    x = x * 2;                          /* modifie la COPIE */
    printf("  dans la fonction : x = %d\n", x);
}

int main(void)
{
    int n = 10;
    tenter_de_doubler(n);
    printf("dans main : n = %d (inchangé !)\n", n);
    return 0;
}
```

Résultat de l'exécution

```
  dans la fonction : x = 20
dans main : n = 10 (inchangé !)
```

Comment écrire une fonction qui modifie vraiment `n` ? Réponse dans la partie 2, avec les **pointeurs**.

9.4 Tableaux en paramètre

Un tableau passé à une fonction n'est **pas copié** : la fonction travaille sur l'original. Comme elle ne connaît pas sa taille, on la transmet en paramètre :

```c
#include <stdio.h>

void afficher(int t[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", t[i]);
    printf("\n");
}

void mettre_a_zero(int t[], int n)
{
    for (int i = 0; i < n; i++)
        t[i] = 0;
}

int main(void)
{
    int notes[4] = {12, 15, 8, 20};
    afficher(notes, 4);
    mettre_a_zero(notes, 4);
    afficher(notes, 4);          /* l'original a bien été modifié */
    return 0;
}
```

Résultat de l'exécution

```
12 15 8 20
0 0 0 0
```

Pourquoi cette différence avec les `int` ? Parce qu'en réalité, on transmet **l'adresse** du tableau. Encore les pointeurs (section 18).

9.5 La récursivité

Une fonction **récursive** s'appelle elle-même. Il lui faut toujours un **cas de base** qui arrête les appels.

```c
#include <stdio.h>

int factorielle(int n)
{
    if (n <= 1)                      /* cas de base : on s'arrête */
        return 1;
    return n * factorielle(n - 1);   /* appel récursif sur un problème plus petit */
}

int main(void)
{
    for (int i = 0; i <= 6; i++)
        printf("%d! = %d\n", i, factorielle(i));
    return 0;
}
```

Résultat de l'exécution

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
```

`factorielle(4)` = `4 * factorielle(3)` = `4 * 3 * factorielle(2)` = `4 * 3 * 2 * factorielle(1)` = `4 * 3 * 2 * 1` = 24. Sans cas de base, les appels ne s'arrêtent jamais et le programme plante (*stack overflow*).

9.6 Portée et durée de vie des variables

```c
#include <stdio.h>

int total = 0;                   /* GLOBALE : visible partout, vit tout le programme */

void compter(void)
{
    int locale = 0;              /* LOCALE : recréée à chaque appel */
    static int appels = 0;       /* STATIC : garde sa valeur entre les appels */
    locale++;
    appels++;
    total++;
    printf("locale = %d, appels = %d, total = %d\n", locale, appels, total);
}

int main(void)
{
    compter();
    compter();
    compter();
    return 0;
}
```

Résultat de l'exécution

```
locale = 1, appels = 1, total = 1
locale = 1, appels = 2, total = 2
locale = 1, appels = 3, total = 3
```

Type de variable	Visible dans	Durée de vie
locale	son bloc `{ }` uniquement	jusqu'à la fin du bloc
locale `static`	son bloc uniquement	tout le programme (garde sa valeur)
globale	tout le fichier	tout le programme

Astuce

Évite les variables globales : elles rendent le code difficile à suivre. Passe plutôt les valeurs en paramètres.

10. Les structures

10.1 Regrouper des données différentes

Une structure regroupe plusieurs variables (de types différents) sous un même nom :

```c
#include <stdio.h>
#include <string.h>

struct Etudiant {
    char nom[30];
    int age;
    double moyenne;
};

int main(void)
{
    struct Etudiant e1 = {"Awa", 21, 14.5};             /* dans l'ordre des champs */
    struct Etudiant e2 = {.age = 22, .moyenne = 16.0,
                          .nom = "Moussa"};             /* par nom de champ (C99) */
    struct Etudiant e3;                                 /* puis remplissage champ par champ */
    strcpy(e3.nom, "Ibra");                             /* pas de e3.nom = "Ibra" ! */
    e3.age = 20;
    e3.moyenne = 11.75;

    printf("%s, %d ans, %.2f\n", e1.nom, e1.age, e1.moyenne);
    printf("%s, %d ans, %.2f\n", e2.nom, e2.age, e2.moyenne);
    printf("%s, %d ans, %.2f\n", e3.nom, e3.age, e3.moyenne);
    return 0;
}
```

Résultat de l'exécution

```
Awa, 21 ans, 14.50
Moussa, 22 ans, 16.00
Ibra, 20 ans, 11.75
```

On accède à un champ avec le **point** : `variable.champ`.

10.2 `typedef` : donner un nom court

```c
typedef struct {
    double x;
    double y;
} Point;                /* on écrit maintenant "Point" au lieu de "struct ..." */

Point a = {1.5, -2.0};
```

10.3 Tableau de structures et copie

```c
#include <stdio.h>

typedef struct {
    char nom[30];
    double moyenne;
} Etudiant;

int main(void)
{
    Etudiant classe[3] = {
        {"Awa", 14.5}, {"Moussa", 16.0}, {"Sokhna", 12.25}
    };

    int meilleur = 0;
    for (int i = 1; i < 3; i++)
        if (classe[i].moyenne > classe[meilleur].moyenne)
            meilleur = i;
    printf("Major : %s (%.2f)\n", classe[meilleur].nom, classe[meilleur].moyenne);

    Etudiant copie = classe[0];     /* une structure SE COPIE avec = */
    copie.moyenne = 20;             /* (contrairement à un tableau)  */
    printf("Original : %.2f, copie : %.2f\n", classe[0].moyenne, copie.moyenne);
    return 0;
}
```

Résultat de l'exécution

```
Major : Moussa (16.00)
Original : 14.50, copie : 20.00
```

À retenir

- Accès à un champ : `e.champ`. Avec un pointeur vers la structure : `p->champ` (partie 2).
- Une structure **se copie** avec `=` (tous les champs, même les tableaux qu'elle contient).
- Passée à une fonction, elle est **copiée** (passage par valeur) : la fonction ne peut pas modifier l'original… sauf avec un pointeur.

Teste-toi

1. Que valent `17 / 5`, `17 % 5` et `17 / 5.0` ?
2. Pourquoi `scanf(" %c", &c)` a-t-il un espace avant `%c` ?
3. Combien de cases faut-il pour stocker la chaîne `"Dakar"` ?
4. Quelle différence entre `while` et `do ... while` ?
5. Un tableau `int t[10]` : quels sont le premier et le dernier indice valides ?
6. Pourquoi une fonction `void f(int x) { x = 0; }` ne modifie-t-elle pas la variable passée ?

PARTIE 2

Les pointeurs

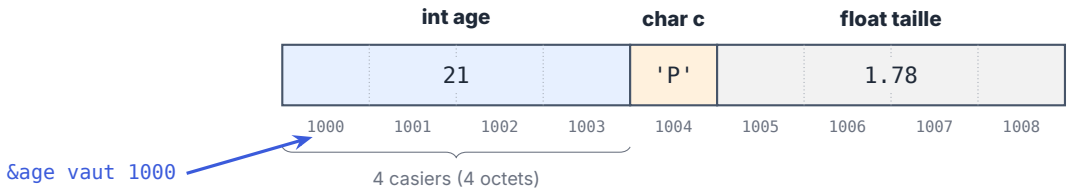
Le cœur du C. On reprend tout depuis le début, doucement : ce qu'est une adresse, ce que contient un pointeur, et pourquoi on en a besoin partout (fonctions, tableaux, chaînes, structures… et malloc).

11. La mémoire et les adresses

11.1 La mémoire, une longue rue de casiers

Analogie

Imagine la mémoire de ton ordinateur comme une **rue immense bordée de casiers**. Chaque casier contient **1 octet** et porte un **numéro unique** : son **adresse**. Quand tu déclares une variable, le système lui réserve un ou plusieurs casiers consécutifs. L'adresse de la variable, c'est le numéro de **son premier casier**.



Chaque variable a donc **quatre caractéristiques** : un nom, un type (donc une taille), une valeur… et une **adresse**. (Les adresses du schéma sont simplifiées ; les vraies sont de grands nombres écrits en hexadécimal.)

11.2 L'opérateur & : « l'adresse de »

`&variable` donne l'adresse de la variable. On l'affiche avec le format `%p` :

```c
#include <stdio.h>

int main(void)
{
    int age = 21;
    char c = 'P';
    double taille = 1.78;

    printf("age    vaut %-5d, occupe %zu octets, rangé à l'adresse %p\n",
           age, sizeof(age), (void *)&age);
    printf("c      vaut %-5c, occupe %zu octet,  rangé à l'adresse %p\n",
```

```
           c, sizeof(c), (void *)&c);
    printf("taille vaut %-5.2f, occupe %zu octets, rangé à l'adresse %p\n",
           taille, sizeof(taille), (void *)&taille);
    return 0;
}
```

Résultat (les adresses changent à chaque exécution)

```
age    vaut 21   , occupe 4 octets, rangé à l'adresse 0x7ffca99da95c
c      vaut P    , occupe 1 octet,  rangé à l'adresse 0x7ffca99da95b
taille vaut 1.78 , occupe 8 octets, rangé à l'adresse 0x7ffca99da960
```

Une adresse, c'est juste un **nombre** (affiché en hexadécimal, d'où le `0x`). Le `(void *)` devant est la conversion attendue par `%p` ; on y reviendra.

À retenir

`&x` = « **où habite** `x` ? » → renvoie son adresse. Tu l'utilises déjà depuis le début avec `scanf("%d", &x)` : tu donnes à `scanf` l'adresse de `x` pour qu'il sache **où** écrire.

12. Qu'est-ce qu'un pointeur ?

12.1 Une variable qui contient une adresse

Un **pointeur** est une variable comme les autres… sauf que sa valeur est une **adresse**.

Analogie

Une variable normale, c'est une **maison** avec quelque chose dedans. Un pointeur, c'est un **papier sur lequel est écrite l'adresse d'une maison**. Le papier ne contient pas la maison ; il te dit **où aller** pour la trouver.



12.2 Déclarer, initialiser, utiliser

```c
#include <stdio.h>

int main(void)
{
    int a = 10;
    int *p;          /* p est un pointeur vers un int (pour l'instant, vide) */
    p = &a;          /* p reçoit l'adresse de a : p "pointe vers" a          */

    printf("a   = %d\n", a);               /* la valeur de a                */
    printf("&a  = %p\n", (void *)&a);      /* l'adresse de a                */
    printf("p   = %p\n", (void *)p);       /* le contenu de p : même adresse */
    printf("*p  = %d\n", *p);              /* la valeur À l'adresse p       */
    printf("&p  = %p\n", (void *)&p);      /* p a aussi sa propre adresse ! */
    return 0;
}
```

Résultat (les adresses changent à chaque exécution)

```
a   = 10
&a  = 0x7fffe6a930ec
p   = 0x7fffe6a930ec
*p  = 10
&p  = 0x7fffe6a930f0
```

Décortiquons :

Expression	Se lit	Valeur ici
`a`	la valeur de `a`	`10`
`&a`	l'adresse de `a`	une adresse (ex. `0x7ffc...`)
`p`	le contenu de `p`	**la même** adresse que `&a`
`*p`	« ce qu'il y a à l'adresse contenue dans `p` »	`10`
`&p`	l'adresse de la variable `p` elle-même	une **autre** adresse

L'opération `*p` s'appelle le **déréférencement** : on suit la flèche pour aller lire (ou écrire) dans la boîte pointée.

12.3 Les différentes façons d'écrire la déclaration

```c
int *p;          /* style le plus courant en C             */
int* p;          /* équivalent                              */
int * p;         /* équivalent                              */
int *p = &a;     /* déclaration + initialisation directe    */
int *p = NULL;   /* pointeur qui ne pointe vers rien (section 16) */
```

Piège classique

L'étoile s'attache au **nom**, pas au type. Dans `int* a, b;`, seul `a` est un pointeur ; `b` est un simple `int` :

```c
#include <stdio.h>

int main(void)
{
    int* a, b;            /* piège : b n'est PAS un pointeur */
    printf("sizeof(a) = %zu (pointeur)\n", sizeof(a));
    printf("sizeof(b) = %zu (int)\n", sizeof(b));

    int *c, *d;           /* la bonne façon : une étoile par pointeur */
    printf("sizeof(c) = %zu, sizeof(d) = %zu\n", sizeof(c), sizeof(d));
    (void)a; (void)b; (void)c; (void)d;
    return 0;
}
```

Résultat de l'exécution

```
sizeof(a) = 8 (pointeur)
sizeof(b) = 4 (int)
sizeof(c) = 8, sizeof(d) = 8
```

C'est pour ça qu'on préfère écrire `int *a` : l'étoile collée au nom rappelle à quoi elle s'applique.

13. Modifier une variable à travers un pointeur

C'est **la** raison d'être des pointeurs : `*p` n'est pas une copie, c'est **la boîte elle-même**. Écrire dans `*p`, c'est écrire dans `a`.

```c
#include <stdio.h>

int main(void)
{
    int a = 10;
    int *p = &a;

    *p = 20;                                    /* écrit DANS a, via p */
    printf("Après *p = 20 : a = %d\n", a);

    a = 30;                                     /* et dans l'autre sens */
    printf("Après a = 30  : *p = %d\n", *p);

    *p = *p + 5;                                /* lire et écrire via p */
    printf("Après *p = *p + 5 : a = %d\n", a);

    (*p)++;                                     /* incrémente a (parenthèses obligatoires) */
    printf("Après (*p)++ : a = %d\n", a);
    return 0;
}
```

Résultat de l'exécution

```
Après *p = 20 : a = 20
Après a = 30  : *p = 30
Après *p = *p + 5 : a = 35
Après (*p)++ : a = 36
```

13.1 Plusieurs pointeurs, une seule variable

```c
#include <stdio.h>

int main(void)
{
    int score = 100;
    int *p1 = &score;
    int *p2 = &score;        /* deux papiers avec la même adresse */
    int *p3 = p1;            /* copier un pointeur = copier l'adresse */

    *p2 = 250;
    printf("score = %d, *p1 = %d, *p2 = %d, *p3 = %d\n", score, *p1, *p2, *p3);
    printf("p1 == p3 ? %s\n", (p1 == p3) ? "oui, même adresse" : "non");
    return 0;
}
```

Résultat de l'exécution

```
score = 250, *p1 = 250, *p2 = 250, *p3 = 250
p1 == p3 ? oui, même adresse
```

13.2 Changer la cible d'un pointeur

Un pointeur n'est pas « marié » à une variable : on peut lui faire pointer ailleurs.

```c
#include <stdio.h>

int main(void)
{
    int a = 1, b = 2;
    int *p = &a;
```

```c
    printf("p pointe vers a : *p = %d\n", *p);

    p = &b;                 /* sans étoile : on change l'ADRESSE stockée */
    printf("p pointe vers b : *p = %d\n", *p);

    *p = 99;                /* avec étoile : on change la VALEUR pointée */
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Résultat de l'exécution

```
p pointe vers a : *p = 1
p pointe vers b : *p = 2
a = 1, b = 99
```

À retenir

- `p = ...` (sans étoile) : on change **vers où** pointe `p`.
- `*p = ...` (avec étoile) : on change **ce qu'il y a** à l'endroit pointé.

14. * et & : ne plus jamais les confondre

Le même symbole a plusieurs sens selon l'endroit où il apparaît. C'est la source n° 1 de confusion :

Écriture	Contexte	Signification
`int *p;`	dans une **déclaration**	« `p` est un pointeur vers un `int` »
`*p`	dans une **expression**	« le contenu à l'adresse `p` » (déréférencement)
`a * b`	entre deux valeurs	multiplication
`&a`	devant une variable	« l'adresse de `a` »
`a & b`	entre deux valeurs	ET bit à bit
`a && b`	entre deux conditions	ET logique

& et * sont des opérations **inverses** l'une de l'autre :

```c
#include <stdio.h>

int main(void)
{
    int a = 7;
    int *p = &a;
    printf("*(&a) = %d   (l'adresse de a, puis son contenu : a)\n", *(&a));
    printf("&(*p) == p ? %s\n", (&(*p) == p) ? "oui" : "non");
    return 0;
}
```

Résultat de l'exécution

```
*(&a) = 7   (l'adresse de a, puis son contenu : a)
&(*p) == p ? oui
```

Analogie

- &x : « **Où habites-tu ?** » → on obtient une adresse.
- *p : « **Toc toc**, je vais à l'adresse notée sur `p` et je regarde qui est là. »

15. Le type d'un pointeur et sa taille

15.1 Tous les pointeurs ont la même taille

Un pointeur stocke une adresse ; sur une machine 64 bits, une adresse fait **8 octets**, quel que soit le type pointé :

```c
#include <stdio.h>

int main(void)
{
    printf("sizeof(char *)   = %zu\n", sizeof(char *));
    printf("sizeof(int *)    = %zu\n", sizeof(int *));
    printf("sizeof(double *) = %zu\n", sizeof(double *));
    printf("sizeof(int **)   = %zu\n", sizeof(int **));
    return 0;
}
```

Résultat de l'exécution

```
sizeof(char *)   = 8
sizeof(int *)    = 8
sizeof(double *) = 8
sizeof(int **)   = 8
```

15.2 Alors, à quoi sert le type ?

Le type indique **comment lire ce qu'il y a au bout de la flèche** : combien d'octets lire, et comment les interpréter. Un `int *` lit 4 octets et les interprète comme un entier ; un `char *` lit 1 octet. Le type sert aussi à l'**arithmétique des pointeurs** (section 18).

Pour s'en convaincre, lisons un `int` octet par octet avec un `unsigned char *` :

```c
#include <stdio.h>

int main(void)
{
    int n = 0x12345678;                          /* 4 octets : 12 34 56 78 */
    unsigned char *octet = (unsigned char *)&n;  /* même adresse, autre type */

    printf("Lu comme un int  : 0x%x\n", n);
    for (int i = 0; i < 4; i++)
        printf("octet %d (adresse %p) : %02x\n", i, (void *)(octet + i), octet[i]);
    return 0;
}
```

Résultat de l'exécution

```
Lu comme un int  : 0x12345678
octet 0 (adresse 0x7fffae5f11e8) : 78
octet 1 (adresse 0x7fffae5f11e9) : 56
octet 2 (adresse 0x7fffae5f11ea) : 34
octet 3 (adresse 0x7fffae5f11eb) : 12
```

Astuce

Les octets apparaissent « à l'envers » (`78 56 34 12`) : les processeurs Intel/AMD sont **little-endian**, ils rangent l'octet de poids faible en premier. Ce détail compte en réseau, en analyse de fichiers binaires et en sécurité (lecture de dumps mémoire).

15.3 Le pointeur générique `void *`

`void *` est un pointeur « sans type » : il peut recevoir n'importe quelle adresse, mais on ne peut pas le déréférencer directement (on ne sait pas combien d'octets lire). Tu le retrouveras dans `malloc`, qui renvoie justement un `void *`. C'est aussi pour ça que `%p` attend un `(void *)`.

16. `NULL` et les pointeurs non initialisés

16.1 Le pointeur non initialisé : un papier griffonné au hasard

Comme toute variable locale, un pointeur non initialisé contient **n'importe quoi**, donc une adresse au hasard. Écrire à travers lui, c'est écrire n'importe où dans la mémoire. Heureusement, gcc le repère :

```c
#include <stdio.h>

int main(void)
{
    int *p;            /* adresse aléatoire ! */
    *p = 5;            /* écrit on ne sait où : comportement indéfini */
    printf("%d\n", *p);
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘main’:
prog.c:6:8: warning: ‘p’ is used uninitialized [-Wuninitialized]
    6 |     *p = 5;            /* écrit on ne sait où : comportement indéfini */
      |     ~~~^~~
prog.c:5:10: note: ‘p’ was declared here
    5 |     int *p;            /* adresse aléatoire ! */
      |          ^
```

On n'exécute pas ce programme : selon les cas, il plante, il corrompt d'autres variables ou il semble marcher… jusqu'au jour où il ne marche plus.

16.2 `NULL` : « ne pointe vers rien »

`NULL` est une adresse spéciale (valeur 0) qui signifie « ce pointeur ne pointe vers **rien** ». Règle d'or : un pointeur qui ne pointe pas encore vers quelque chose de valide doit valoir `NULL`, et on **teste** avant d'utiliser.

```c
#include <stdio.h>

void afficher(int *p)
{
    if (p == NULL) {                /* ou : if (!p) */
        printf("Pointeur NULL : rien à afficher\n");
        return;
    }
    printf("Valeur pointée : %d\n", *p);
}

int main(void)
{
    int x = 42;
    int *p = NULL;           /* initialisé proprement */
    afficher(p);
    p = &x;
    afficher(p);
    return 0;
}
```

Résultat de l'exécution

```
Pointeur NULL : rien à afficher
Valeur pointée : 42
```

Et si on déréférence `NULL` quand même ? Le système arrête immédiatement le programme :

```c
#include <stdio.h>

int main(void)
{
    int *p = NULL;
    printf("Avant le crash...\n");
    printf("%d\n", *p);          /* lecture à l'adresse 0 : interdit */
    printf("Cette ligne ne s'affichera jamais\n");
    return 0;
}
```

Résultat de l'exécution

```
Avant le crash...
Segmentation fault (core dumped)
```

À retenir

La fameuse « **Segmentation fault** » veut dire : « ton programme a essayé d'accéder à une zone mémoire qui ne lui appartient pas ». Neuf fois sur dix, c'est un pointeur `NULL`, non initialisé ou déjà libéré. Un crash immédiat sur `NULL` est un *bon* crash : il est facile à trouver. D'où la règle : `int *p = NULL;` plutôt que `int *p;`.

17. Pointeurs et fonctions : le passage par adresse

17.1 Le problème : la fonction travaille sur des copies

On veut une fonction qui échange deux variables. Première tentative, avec des `int` :

```c
#include <stdio.h>

void echanger(int x, int y)          /* x et y sont des COPIES */
{
    int tmp = x;
    x = y;
    y = tmp;
    printf("  dans echanger : x = %d, y = %d\n", x, y);
}

int main(void)
{
    int a = 3, b = 9;
    echanger(a, b);
    printf("dans main      : a = %d, b = %d  (raté !)\n", a, b);
    return 0;
}
```

Résultat de l'exécution

```
  dans echanger : x = 9, y = 3
dans main      : a = 3, b = 9  (raté !)
```

17.2 La solution : passer les adresses

On donne à la fonction **l'adresse** de `a` et de `b`. Avec `*`, elle peut alors écrire directement dans les variables de `main` :

```c
#include <stdio.h>

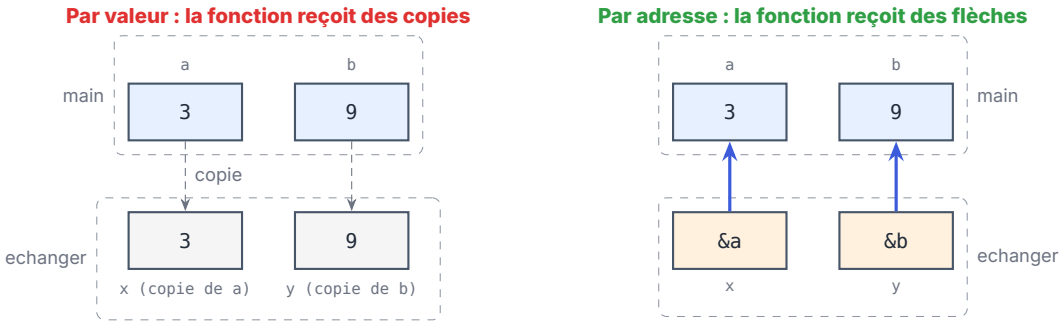
void echanger(int *x, int *y)        /* x et y reçoivent des ADRESSES */
{
    int tmp = *x;                    /* tmp = valeur de a */
    *x = *y;                         /* a = valeur de b   */
```

```c
    *y = tmp;                       /* b = ancienne valeur de a */
}

int main(void)
{
    int a = 3, b = 9;
    echanger(&a, &b);               /* on envoie les adresses */
    printf("a = %d, b = %d  (réussi !)\n", a, b);
    return 0;
}
```

Résultat de l'exécution

```
a = 9, b = 3  (réussi !)
```



Pas à pas, pendant l'appel `echanger(&a, &b)` :

1. `x` reçoit l'adresse de `a`, `y` reçoit l'adresse de `b`.
2. `tmp = *x;` → on va à l'adresse de `a`, on lit 3.
3. `*x = *y;` → on lit 9 chez `b` et on l'écrit **chez `a`**.
4. `*y = tmp;` → on écrit 3 **chez `b`**. Les variables de `main` ont réellement changé.

À retenir

Pour qu'une fonction **modifie** une variable de l'appelant : le paramètre est un **pointeur** (`int *x`), l'appel passe une **adresse** (`&a`), et dans la fonction on écrit avec **`*x = ...`**. C'est exactement ce que fait `scanf("%d", &n)`.

17.3 Renvoyer plusieurs résultats

`return` ne renvoie qu'une valeur. Avec des pointeurs, une fonction peut en « renvoyer » autant qu'on veut :

```c
#include <stdio.h>

void division(int a, int b, int *quotient, int *reste)
{
    *quotient = a / b;
    *reste = a % b;
}

int main(void)
{
    int q, r;
    division(17, 5, &q, &r);
    printf("17 = 5 x %d + %d\n", q, r);
    return 0;
}
```

Résultat de l'exécution

```
17 = 5 x 3 + 2
```

Variante très courante : la fonction **renvoie un code d'erreur** et écrit le vrai résultat via un pointeur :

```c
#include <stdio.h>

/* renvoie 1 si la division est possible, 0 sinon */
int diviser(double a, double b, double *resultat)
{
    if (b == 0)
        return 0;
    *resultat = a / b;
    return 1;
}

int main(void)
{
    double r;
    if (diviser(10, 4, &r))
        printf("10 / 4 = %.2f\n", r);
    if (!diviser(10, 0, &r))
        printf("10 / 0 : impossible\n");
    return 0;
}
```

Résultat de l'exécution

```
10 / 4 = 2.50
10 / 0 : impossible
```

17.4 L'oubli du & dans `scanf`, enfin expliqué

`scanf("%d", n)` envoie la **valeur** de `n` (n'importe quoi) au lieu de son adresse : `scanf` écrirait à une adresse au hasard. gcc le détecte :

```c
#include <stdio.h>

int main(void)
{
    int n;
    scanf("%d", n);           /* il manque le & */
    printf("%d\n", n);
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘main’:
prog.c:6:13: warning: format ‘%d’ expects argument of type ‘int *’, but argument 2 has type
    ‘int’ [-Wformat=]
    6 |     scanf("%d", n);           /* il manque le & */
      |            ~^   ~
      |             |   |
      |             |   int
      |             int *
prog.c:6:5: warning: ‘n’ is used uninitialized [-Wuninitialized]
    6 |     scanf("%d", n);           /* il manque le & */
      |     ^~~~~~~~~~~~~
prog.c:5:9: note: ‘n’ was declared here
    5 |     int n;
      |         ^
```

17.5 Piège mortel : renvoyer l'adresse d'une variable locale

Les variables locales d'une fonction **disparaissent** quand la fonction se termine. Renvoyer leur adresse, c'est donner l'adresse d'une maison déjà démolie :

```c
#include <stdio.h>

int *creer_nombre(void)
{
    int x = 42;          /* x vit seulement pendant l'appel */
    return &x;           /* adresse d'une variable qui va mourir */
}

int main(void)
{
    int *p = creer_nombre();
    printf("On essaie de lire la valeur...\n");
    printf("Valeur : %d\n", *p);
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘creer_nombre’:
prog.c:6:12: warning: function returns address of local variable [-Wreturn-local-addr]
    6 |     return &x;           /* adresse d'une variable qui va mourir */
      |            ^~
```

Résultat de l'exécution

```
On essaie de lire la valeur...
Segmentation fault (core dumped)
```

gcc prévient, et en plus il remplace ce pointeur dangereux par `NULL`, d'où le crash. La solution pour créer une donnée qui **survit** à la fonction ? L'allocation dynamique avec `malloc` (partie 3).

18. Pointeurs et tableaux

18.1 Le nom d'un tableau, c'est l'adresse de sa première case

```c
#include <stdio.h>

int main(void)
{
    int t[4] = {10, 20, 30, 40};

    printf("t     = %p\n", (void *)t);
    printf("&t[0] = %p   <- identique\n", (void *)&t[0]);
    printf("&t[1] = %p   <- +4 octets (un int)\n", (void *)&t[1]);
    printf("&t[2] = %p   <- +8\n", (void *)&t[2]);
    printf("&t[3] = %p   <- +12\n", (void *)&t[3]);
    return 0;
}
```

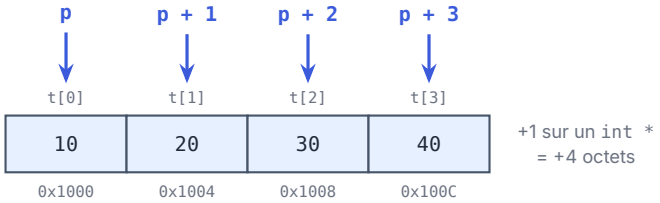
Résultat (les adresses changent à chaque exécution)

```
t     = 0x7fff8a868150
&t[0] = 0x7fff8a868150   <- identique
&t[1] = 0x7fff8a868154   <- +4 octets (un int)
&t[2] = 0x7fff8a868158   <- +8
&t[3] = 0x7fff8a86815c   <- +12
```

Les cases sont collées, chacune décalée de `sizeof(int)` = 4 octets. Et `t` (sans crochets) vaut l'adresse de `t[0]`.

18.2 L'arithmétique des pointeurs

Si `p` pointe vers une case d'un tableau, `p + 1` pointe vers la **case suivante**. Le compilateur avance automatiquement de `sizeof(type)` octets, pas d'un seul octet :



D'où l'équivalence fondamentale : **`t[i]` est exactement `*(t + i)`**.

```c
#include <stdio.h>

int main(void)
{
    int t[5] = {10, 20, 30, 40, 50};
    int *p = t;                         /* équivaut à p = &t[0] */

    printf("t[2]     = %d\n", t[2]);
    printf("*(t + 2) = %d\n", *(t + 2));
    printf("*(p + 2) = %d\n", *(p + 2));
    printf("p[2]     = %d   (on peut utiliser [] sur un pointeur)\n", p[2]);

    p = p + 3;                          /* p pointe maintenant sur t[3] */
    printf("Après p = p + 3 : *p = %d, p[-1] = %d\n", *p, p[-1]);
    printf("Distance entre p et t : %td cases\n", p - t);
    return 0;
}
```

Résultat de l'exécution

```
t[2]     = 30
*(t + 2) = 30
*(p + 2) = 30
p[2]     = 30   (on peut utiliser [] sur un pointeur)
Après p = p + 3 : *p = 40, p[-1] = 30
Distance entre p et t : 3 cases
```

Astuce

La soustraction de deux pointeurs donne un **nombre de cases** (type `ptrdiff_t`, format `%td`). Et comme `t[i]` vaut `*(t + i)` = `*(i + t)`, on peut même écrire `i[t]`… ce qui est légal, inutile, et idéal pour piéger un camarade.

18.3 Parcourir un tableau avec un pointeur

Deux styles équivalents pour le même parcours :

```c
#include <stdio.h>

int main(void)
{
    int t[5] = {3, 1, 4, 1, 5};
    int n = 5;

    /* Style 1 : avec un indice */
    for (int i = 0; i < n; i++)
        printf("%d ", t[i]);
    printf("\n");
```

```c
    /* Style 2 : avec un pointeur qui avance de case en case */
    for (int *p = t; p < t + n; p++)
        printf("%d ", *p);
    printf("\n");

    /* Doubler chaque case via le pointeur */
    for (int *p = t; p < t + n; p++)
        *p = *p * 2;
    for (int i = 0; i < n; i++)
        printf("%d ", t[i]);
    printf("\n");
    return 0;
}
```

Résultat de l'exécution

```
3 1 4 1 5
3 1 4 1 5
6 2 8 2 10
```

18.4 *p++ ou (*p)++ ?

Deux écritures qui se ressemblent, deux effets différents :

```c
#include <stdio.h>

int main(void)
{
    int t[3] = {10, 20, 30};
    int *p = t;

    (*p)++;                 /* incrémente la VALEUR pointée : t[0] passe à 11 */
    printf("Après (*p)++ : t[0] = %d, p pointe sur t[0]\n", t[0]);

    int v = *p++;           /* lit *p PUIS avance le pointeur */
    printf("Après v = *p++ : v = %d, *p = %d (p pointe sur t[1])\n", v, *p);
    return 0;
}
```

Résultat de l'exécution

```
Après (*p)++ : t[0] = 11, p pointe sur t[0]
Après v = *p++ : v = 11, *p = 20 (p pointe sur t[1])
```

`*p++` se lit `*(p++)` : on lit la case actuelle puis le pointeur avance. Dans le doute, mets des parenthèses.

18.5 Tableau et pointeur : proches, mais pas identiques

	`int t[5];`	`int *p = t;`
Ce que c'est	5 cases réservées	**une** variable contenant une adresse
`sizeof`	20 (5 × 4 octets)	8 (taille d'une adresse)
Peut changer de cible ?	Non : `t = ...` est interdit	Oui : `p = ...`
Accès aux éléments	`t[i]` ou `*(t + i)`	`p[i]` ou `*(p + i)`

18.6 Ce qui se passe quand on passe un tableau à une fonction

Un tableau passé en paramètre est **converti en pointeur** vers sa première case. La fonction reçoit une adresse, pas une copie : c'est pour ça qu'elle peut le modifier, et pour ça qu'elle ne connaît pas sa taille.

```c
#include <stdio.h>

void taille_dans_fonction(int t[])     /* t[] ici est en réalité un int * */
{
    printf("Dans la fonction : sizeof(t) = %zu\n", sizeof(t));
}

int main(void)
{
    int t[5] = {1, 2, 3, 4, 5};
    printf("Dans main        : sizeof(t) = %zu\n", sizeof(t));
    taille_dans_fonction(t);
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘taille_dans_fonction’:
prog.c:5:58: warning: ‘sizeof’ on array function parameter ‘t’ will return size of ‘int *’
    [-Wsizeof-array-argument]
    5 |     printf("Dans la fonction : sizeof(t) = %zu\n", sizeof(t));
      |                                                          ^
prog.c:3:31: note: declared here
    3 | void taille_dans_fonction(int t[])     /* t[] ici est en réalité un int * */
      |                           ~~~~^~~
```

Résultat de l'exécution

```
Dans main        : sizeof(t) = 20
Dans la fonction : sizeof(t) = 8
```

À retenir

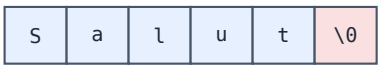
- `void f(int t[])` et `void f(int *t)` sont **strictement identiques**.
- Dans une fonction, `sizeof(t)` donne la taille d'un pointeur (8), pas celle du tableau : il faut **toujours passer la taille** en paramètre : `void f(int *t, int n)`.

19. Pointeurs et chaînes de caractères

19.1 `char s[]` ou `char *s` : la différence qui fait planter

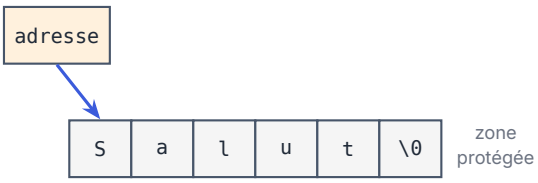
`char s1[] = "Salut";`

un tableau à toi, sur la pile : **modifiable**



`char *s2 = "Salut";`

un pointeur vers un texte **en lecture seule**




```c
#include <stdio.h>

int main(void)
{
    char s1[] = "Salut";      /* copie du texte dans un tableau à toi */
    char *s2 = "Salut";       /* pointe vers le texte d'origine, protégé */

    s1[0] = 's';              /* OK */
```

```
    printf("s1 = %s\n", s1);

    printf("On tente de modifier s2...\n");
    s2[0] = 's';              /* écriture dans une zone en lecture seule */
    printf("s2 = %s\n", s2);
    return 0;
}
```

Résultat de l'exécution

```
s1 = salut
On tente de modifier s2...
Segmentation fault (core dumped)
```

Pour se protéger, on déclare les pointeurs vers des textes littéraux avec `const` : le compilateur refuse alors la modification **avant** l'exécution.

```c
int main(void)
{
    const char *s2 = "Salut";
    s2[0] = 's';
    return 0;
}
```

Ce que dit le compilateur : erreur, pas d'exécutable

```
prog.c: In function ‘main’:
prog.c:4:11: error: assignment of read-only location ‘*s2’
    4 |     s2[0] = 's';
      |           ^
```

19.2 Parcourir une chaîne avec un pointeur

Une chaîne se termine par `'\0'` : on avance le pointeur jusqu'à le rencontrer. Voici comment on pourrait réécrire `strlen` :

```c
#include <stdio.h>
#include <stddef.h>

size_t ma_longueur(const char *s)
{
    const char *p = s;          /* p part du début */
    while (*p != '\0')          /* tant qu'on n'est pas sur la fin... */
        p++;                    /* ... on avance d'un caractère */
    return p - s;               /* distance parcourue = longueur */
}

int main(void)
{
    printf("ma_longueur(\"Dakar\") = %zu\n", ma_longueur("Dakar"));
    printf("ma_longueur(\"\")      = %zu\n", ma_longueur(""));
    return 0;
}
```

Résultat de l'exécution

```
ma_longueur("Dakar") = 5
ma_longueur("")      = 0
```

19.3 Tableau de chaînes et arguments du `main`

Un tableau de pointeurs vers des chaînes est très pratique :

```c
#include <stdio.h>

int main(void)
{
    const char *jours[] = {"Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi"};
    int n = sizeof(jours) / sizeof(jours[0]);

    for (int i = 0; i < n; i++)
        printf("%d. %s (première lettre : %c)\n", i + 1, jours[i], jours[i][0]);
    return 0;
}
```

Résultat de l'exécution

```
1. Lundi (première lettre : L)
2. Mardi (première lettre : M)
3. Mercredi (première lettre : M)
4. Jeudi (première lettre : J)
5. Vendredi (première lettre : V)
```

C'est exactement la forme de `argv` dans `int main(int argc, char *argv[])` : un tableau de chaînes contenant les mots tapés après le nom du programme.

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++)
        printf("argv[%d] = \"%s\"\n", i, argv[i]);
    return 0;
}
```

Résultat de l'exécution

```
$ ./prog Pape 21 Dakar
argc = 4
argv[0] = "./prog"
argv[1] = "Pape"
argv[2] = "21"
argv[3] = "Dakar"
```

20. Pointeurs et structures

20.1 L'opérateur flèche `->`

Avec un pointeur vers une structure, on accède aux champs avec `->`. C'est un raccourci : `p->age` signifie `(*p).age`.

```c
#include <stdio.h>

typedef struct {
    char nom[30];
    int age;
} Etudiant;

int main(void)
{
    Etudiant e = {"Awa", 21};
    Etudiant *p = &e;

    printf("e.nom     = %s\n", e.nom);      /* via la variable          */
```

```c
    printf("(*p).nom  = %s\n", (*p).nom);   /* via le pointeur (lourd)  */
    printf("p->nom    = %s\n", p->nom);     /* via le pointeur (pratique) */

    p->age = 22;                            /* modifie e.age */
    printf("e.age     = %d\n", e.age);
    return 0;
}
```

Résultat de l'exécution

```
e.nom     = Awa
(*p).nom  = Awa
p->nom    = Awa
e.age     = 22
```

Piège classique

`*p.age` ne marche pas : le `.` est prioritaire sur `*`, donc le compilateur comprend `*(p.age)`. Il faut `(*p).age`... ou simplement `p->age`. Règle simple : **variable → point, pointeur → flèche**.

20.2 Modifier une structure dans une fonction

```c
#include <stdio.h>

typedef struct {
    char nom[30];
    int age;
} Etudiant;

void anniversaire_rate(Etudiant e)   /* reçoit une COPIE */
{
    e.age++;
}

void anniversaire(Etudiant *e)       /* reçoit l'ADRESSE */
{
    e->age++;
}

void afficher(const Etudiant *e)     /* const : lecture seule, et pas de copie */
{
    printf("%s a %d ans\n", e->nom, e->age);
}

int main(void)
{
    Etudiant e = {"Moussa", 20};
    anniversaire_rate(e);
    afficher(&e);
    anniversaire(&e);
    afficher(&e);
    return 0;
}
```

Résultat de l'exécution

```
Moussa a 20 ans
Moussa a 21 ans
```

Même pour **lire** une structure, on passe souvent un pointeur : on évite de copier toute la structure (qui peut être grosse). Le `const` garantit alors que la fonction ne la modifiera pas :

```c
typedef struct { char nom[30]; int age; } Etudiant;
```

```c
void afficher(const Etudiant *e)
{
    e->age = 0;      /* refusé : structure en lecture seule */
}

int main(void) { return 0; }
```

Ce que dit le compilateur : erreur, pas d'exécutable

```
prog.c: In function ‘afficher’:
prog.c:5:12: error: assignment of member ‘age’ in read-only object
    5 |     e->age = 0;      /* refusé : structure en lecture seule */
      |            ^
```

21. `const` et les pointeurs

Avec un pointeur, deux choses peuvent être constantes : la **valeur pointée** ou le **pointeur lui-même**. Astuce de lecture : on lit la déclaration **de droite à gauche**.

Déclaration	Se lit	`*p = ...`	`p = ...`
`const int *p`	p est un pointeur vers un int constant	interdit	autorisé
`int *const p`	p est un pointeur constant vers un int	autorisé	interdit
`const int *const p`	les deux sont constants	interdit	interdit

```c
int main(void)
{
    int a = 1, b = 2;
    const int *p1 = &a;     /* la valeur pointée est protégée */
    int *const p2 = &a;     /* le pointeur est figé sur a     */

    p1 = &b;                /* OK                             */
    *p1 = 5;                /* ERREUR                         */
    *p2 = 5;                /* OK                             */
    p2 = &b;                /* ERREUR                         */
    return 0;
}
```

Ce que dit le compilateur : erreur, pas d'exécutable

```
prog.c: In function ‘main’:
prog.c:8:9: error: assignment of read-only location ‘*p1’
    8 |     *p1 = 5;                /* ERREUR                         */
      |         ^
prog.c:10:8: error: assignment of read-only variable ‘p2’
   10 |     p2 = &b;                /* ERREUR                         */
      |        ^
```

En pratique, le cas le plus utile est `const type *param` dans les fonctions qui **lisent** sans modifier (comme `strlen(const char *s)`).

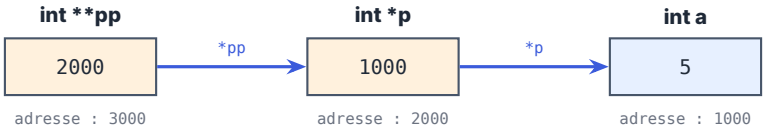
22. Le pointeur de pointeur

22.1 Une adresse vers une adresse

Un pointeur est une variable, il a donc lui aussi une adresse. Un **pointeur de pointeur** (`int **`) stocke l'adresse d'un pointeur.

Analogie

Une **chasse au trésor** : le premier papier (`pp`) te donne l'adresse d'un deuxième papier (`p`), qui te donne l'adresse du trésor (`a`). `*pp` = le deuxième papier ; `**pp` = le trésor.




```c
#include <stdio.h>

int main(void)
{
    int a = 5;
    int *p = &a;
    int **pp = &p;

    printf("a    = %d\n", a);
    printf("*p   = %d\n", *p);
    printf("**pp = %d\n", **pp);
    printf("*pp == p ? %s\n", (*pp == p) ? "oui" : "non");

    **pp = 99;                      /* on suit les deux flèches */
    printf("Après **pp = 99 : a = %d\n", a);
    return 0;
}
```

Résultat de l'exécution

```
a    = 5
*p   = 5
**pp = 5
*pp == p ? oui
Après **pp = 99 : a = 99
```

22.2 À quoi ça sert ? Modifier un pointeur dans une fonction

Même logique que pour `echanger` : pour qu'une fonction modifie un `int`, on lui passe un `int *`. Pour qu'elle modifie un **`int *`**, on lui passe un **`int **`**.

```c
#include <stdio.h>

/* Fait pointer *resultat vers la plus grande des deux variables */
void pointer_vers_max(int **resultat, int *x, int *y)
{
    if (*x > *y)
        *resultat = x;
    else
        *resultat = y;
}

int main(void)
{
    int a = 12, b = 30;
    int *max = NULL;
```

```c
    pointer_vers_max(&max, &a, &b);   /* on passe l'adresse du pointeur */
    printf("Le max vaut %d\n", *max);

    *max = 0;                         /* max pointe vers b : on modifie b */
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Résultat de l'exécution

```
Le max vaut 30
a = 12, b = 0
```

Tu retrouveras ** dans deux situations classiques de la partie 3 : une fonction qui **alloue** un tableau pour l'appelant, et les **matrices dynamiques**. Et `char **argv` est un pointeur de pointeur.

23. Les erreurs fréquentes avec les pointeurs

Erreur	Exemple	Conséquence	Correction
Pointeur non initialisé	`int *p; *p = 5;`	écriture n'importe où	`int *p = NULL;` puis le faire pointer vers une vraie variable
Déréférencer `NULL`	`int *p = NULL; *p = 1;`	Segmentation fault	tester `if (p != NULL)`
Oublier le `&`	`scanf("%d", n);`	écriture à une adresse au hasard	`scanf("%d", &n);`
Étoile en trop / en moins	`p = 5;` au lieu de `*p = 5;`	p pointe vers l'adresse 5	`*p` = la valeur, `p` = l'adresse
Une étoile pour deux	`int* a, b;`	`b` n'est pas un pointeur	`int *a, *b;`
Adresse d'une locale	`return &x;`	pointeur vers une zone morte	`malloc`, ou paramètre pointeur
Comparer des chaînes avec `==`	`if (s == "oui")`	compare les adresses	`strcmp(s, "oui") == 0`
Modifier un littéral	`char *s = "a"; s[0] = 'b';`	Segmentation fault	`char s[] = "a";` ou `const char *`
`sizeof` d'un tableau en paramètre	`sizeof(t)` dans `f(int t[])`	renvoie 8	passer la taille `n`

À retenir

Les pointeurs en 5 phrases :

1. Un pointeur est une variable qui contient une **adresse**.
2. `&x` donne l'adresse de `x` ; `*p` donne ce qu'il y a à l'adresse `p`.
3. Écrire dans `*p`, c'est écrire dans la variable pointée.
4. Pour qu'une fonction modifie une variable, on lui passe son adresse.
5. Un pointeur doit toujours pointer vers quelque chose de valide… ou valoir `NULL`.

Teste-toi

1. Que affiche : `int a = 5; int *p = &a; *p = *p + 3; printf("%d", a);` ?
2. Avec `int t[] = {10, 20, 30}; int *p = t + 1;`, que valent `*p`, `*(p + 1)` et `p[-1]` ?
3. Qu'est-ce qui ne va pas dans `int *p; *p = 10;` ?
4. Quelle est la différence entre `p = &b;` et `*p = b;` ?
5. Sur une machine 64 bits, que valent `sizeof(t)` et `sizeof(p)` avec `double t[10]; double *p = t;` ?
6. Écris le prototype d'une fonction `minmax` qui reçoit un tableau d'`int` et sa taille, et « renvoie » le minimum et le maximum.
7. Pourquoi `void f(int *t)` ne peut-elle pas calculer la taille du tableau avec `sizeof` ?

PARTIE 3

L'allocation dynamique : malloc & compagnie

Réserver de la mémoire pendant l'exécution, autant qu'il en faut, aussi longtemps qu'il le faut… et la rendre. Avec les pointeurs, c'est l'autre pilier du C.

24. Pourquoi l'allocation dynamique ?

Jusqu'ici, toute la mémoire était réservée **automatiquement** : `int t[100];` réserve 100 cases au moment où on entre dans le bloc, et les libère à la sortie. C'est pratique, mais cela pose trois problèmes.

Problème 1 — La taille doit être connue en écrivant le code. Combien d'étudiants dans le fichier ? Combien de caractères dans le nom que l'utilisateur va taper ? On ne le sait qu'à l'exécution. Réserver « large » (`int t[100000]`) gaspille de la mémoire, et ne suffit pas toujours.

Problème 2 — Les variables locales meurent à la fin de la fonction. On l'a vu en section 17 : une fonction ne peut pas renvoyer l'adresse d'une de ses variables locales. Comment écrire alors une fonction qui *crée* un tableau ?

Problème 3 — La zone des variables locales est petite. Sous Linux, elle est limitée à environ 8 Mo. Un gros tableau local fait planter le programme :

```c
#include <stdio.h>

void gros_calcul(void)
{
    int grand[3000000];          /* 3 millions d'int = 12 Mo sur la pile */
    grand[0] = 1;
    printf("grand[0] = %d\n", grand[0]);
}

int main(void)
{
    printf("Appel de gros_calcul...\n");
    gros_calcul();
    printf("Terminé\n");
    return 0;
}
```

Résultat de l'exécution

```
Appel de gros_calcul...
Segmentation fault (core dumped)
```

La solution à ces trois problèmes : demander de la mémoire **pendant l'exécution**, dans une autre zone

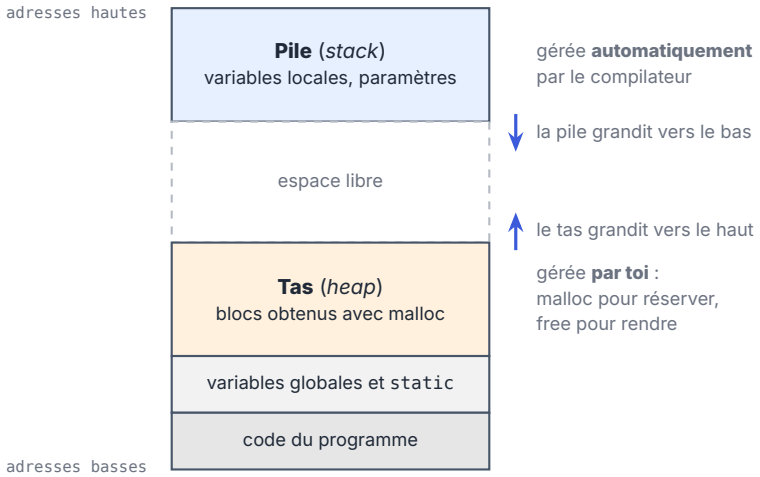
appelée **le tas**, avec `malloc`.

Astuce

Depuis C99, on peut écrire `int t[n];` avec une taille `n` connue à l'exécution (tableau de longueur variable, ou VLA). Mais ce tableau vit toujours sur la pile : il reste limité en taille, meurt à la fin du bloc, et rien ne prévient en cas d'échec. Beaucoup d'enseignants l'interdisent : `malloc` reste la vraie réponse.

25. La pile et le tas

La mémoire d'un programme en cours d'exécution est découpée en zones. Deux nous intéressent :



	Pile (*stack*)	Tas (*heap*)
Ce qu'on y met	variables locales, paramètres	blocs demandés avec `malloc`, `calloc`, `realloc`
Qui réserve et libère	le compilateur, **automatiquement**	**toi** : `malloc` puis `free`
Durée de vie	jusqu'à la fin du bloc `{ }`	jusqu'à ce que tu appelles `free`
Taille disponible	petite (≈ 8 Mo sous Linux)	grande (presque toute la RAM)
Taille connue	à la compilation	à l'exécution
En cas d'oubli	rien à oublier	**fuite mémoire** si tu oublies `free`

Analogie

La pile, c'est le plateau de la cantine : on te le donne à l'entrée, on le reprend à la sortie, tu n'as rien à gérer. Mais il est petit, et tu ne peux pas l'emporter.
Le tas, c'est la location d'un box de stockage : tu choisis la taille au moment de la demande, le box reste à toi même quand tu quittes la pièce, et tu peux confier la clé (l'adresse) à quelqu'un d'autre. En contrepartie, **c'est à toi de rendre la clé** (`free`). Si tu oublies, le box reste occupé pour rien : c'est une **fuite mémoire**.

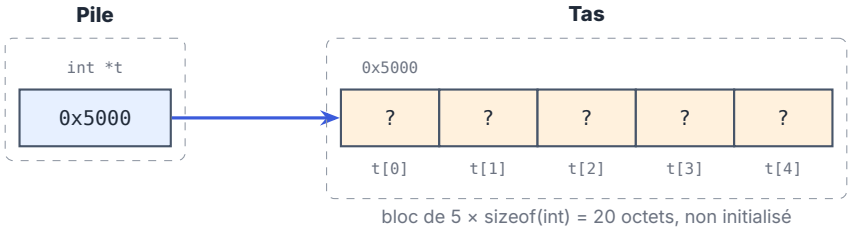
26. `malloc` pas à pas

26.1 Le prototype

```c
#include <stdlib.h>

void *malloc(size_t taille);   /* taille en OCTETS */
```

`malloc` réserve un bloc de `taille` octets dans le tas et renvoie **l'adresse de son premier octet**, ou `NULL` si la mémoire manque. Le contenu du bloc n'est **pas initialisé** (il contient n'importe quoi). Le type de retour `void *` (adresse sans type) se convertit automatiquement vers le type de ton pointeur.



Seul le **pointeur** `t` est sur la pile (8 octets). Les 20 octets du bloc sont dans le tas.

26.2 La recette en 4 étapes

À retenir

1. Allouer — `int *t = malloc(n * sizeof(int));`
2. Vérifier — `if (t == NULL) { /* gérer l'erreur */ }`
3. Utiliser — `t[i] = ...` comme un tableau normal
4. Libérer — `free(t);` puis `t = NULL;`
Chaque `malloc` doit avoir **exactement un** `free` correspondant.

26.3 Premier exemple : un seul entier

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(sizeof(int));        /* 1. allouer 4 octets         */
    if (p == NULL) {                     /* 2. vérifier                 */
        printf("Mémoire insuffisante\n");
        return 1;
    }
    *p = 42;                             /* 3. utiliser                 */
    printf("*p = %d\n", *p);
    free(p);                             /* 4. libérer                  */
    p = NULL;
    return 0;
}
```

Résultat de l'exécution

```
*p = 42
```

Allouer un seul `int` n'a pas grand intérêt en pratique, mais tout le mécanisme est là.

26.4 Deuxième exemple : un tableau

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n = 5;
    int *t = malloc(n * sizeof(int));    /* 5 int = 20 octets */
    if (t == NULL) {
        printf("Mémoire insuffisante\n");
        return 1;
    }

    for (int i = 0; i < n; i++)          /* s'utilise comme un tableau */
        t[i] = (i + 1) * (i + 1);

    for (int i = 0; i < n; i++)
        printf("t[%d] = %d\n", i, t[i]);

    free(t);
    t = NULL;
    return 0;
}
```

Résultat de l'exécution

```
t[0] = 1
t[1] = 4
t[2] = 9
t[3] = 16
t[4] = 25
```

Grâce à l'équivalence `t[i]` ≡ `*(t + i)` (section 18), un bloc alloué s'utilise **exactement comme un tableau**.

26.5 Décortiquer la ligne magique

```
int *t = malloc(n * sizeof(int));
```

Morceau	Signification
`int *t`	une variable pointeur, sur la pile, qui va recevoir l'adresse du bloc
`sizeof(int)`	la taille d'un `int` (4 octets ici) : on ne l'écrit jamais « en dur »
`n * sizeof(int)`	la taille totale du bloc, en **octets**
`malloc(...)`	réserve le bloc dans le tas, renvoie son adresse (ou `NULL`)
`=`	l'adresse (un `void *`) est rangée dans `t`, convertie en `int *`

26.6 Les différentes façons d'écrire la taille

```c
int *t = malloc(5 * sizeof(int));      /* classique et clair                    */
int *t = malloc(5 * sizeof *t);        /* idiome recommandé : suit le type de t  */
int *t = (int *)malloc(5 * sizeof(int)); /* avec conversion explicite : accepté   */
int *t = malloc(20);                   /* À ÉVITER : suppose qu'un int fait 4 octets */
```

Astuce

`sizeof *t` signifie « la taille de ce que pointe `t` ». Si un jour tu changes `int *t` en `double *t`, la taille reste juste automatiquement, alors qu'avec `sizeof(int)` il faudrait penser à corriger. La conversion `(int *)` devant `malloc` est inutile en C (obligatoire en C++) : les deux écritures sont acceptées, utilise celle de ton cours.

27. Un tableau dont la taille est choisie à l'exécution

L'exercice le plus classique : l'utilisateur annonce combien de valeurs il va saisir.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    printf("Combien de notes ? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Nombre invalide\n");
        return 1;
    }

    double *notes = malloc(n * sizeof *notes);   /* exactement n cases */
    if (notes == NULL) {
        printf("Mémoire insuffisante\n");
        return 1;
    }

    printf("Entre les %d notes : ", n);
    for (int i = 0; i < n; i++)
        scanf("%lf", &notes[i]);                  /* &notes[i] : adresse de la case i */

    double somme = 0;
    for (int i = 0; i < n; i++)
        somme += notes[i];
    printf("Moyenne : %.2f\n", somme / n);

    free(notes);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Combien de notes ? 4
Entre les 4 notes : 12 15.5 9 18
Moyenne : 13.62
```

28. `calloc` : allouer et mettre à zéro

```c
void *calloc(size_t nombre, size_t taille_element);
```

`calloc` réserve `nombre × taille_element` octets **et les met tous à zéro**.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n = 6;
    int *compteurs = calloc(n, sizeof *compteurs);   /* 6 int, tous à 0 */
    if (compteurs == NULL)
        return 1;
```

```c
    for (int i = 0; i < n; i++)
        printf("%d ", compteurs[i]);
    printf("\n");

    /* utile pour compter : ici, les chiffres pairs de 2026 */
    int nombre = 2026;
    while (nombre > 0) {
        int chiffre = nombre % 10;
        if (chiffre < n)
            compteurs[chiffre]++;
        nombre /= 10;
    }
    printf("Le chiffre 2 apparaît %d fois dans 2026\n", compteurs[2]);

    free(compteurs);
    return 0;
}
```

Résultat de l'exécution

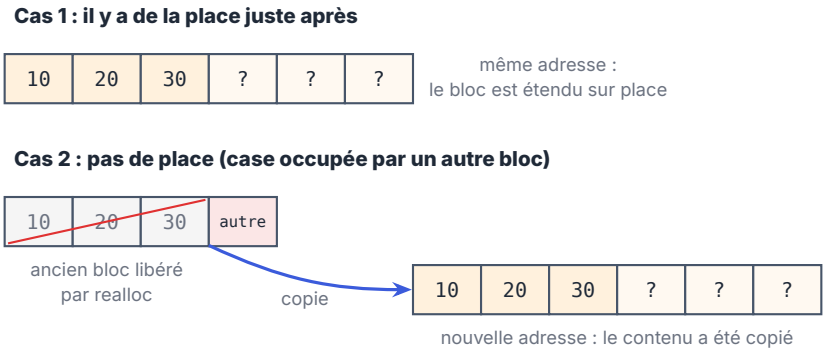
```
0 0 0 0 0 0
Le chiffre 2 apparaît 2 fois dans 2026
```

	`malloc(n * sizeof(int))`	`calloc(n, sizeof(int))`
Arguments	une taille totale	un nombre d'éléments et la taille d'un élément
Contenu initial	**indéterminé** (valeurs aléatoires)	**tout à zéro**
Quand l'utiliser	quand tu remplis tout de suite toutes les cases	compteurs, sommes, tableaux qui doivent partir de 0

29. `realloc` : agrandir ou rétrécir un bloc

```c
void *realloc(void *ancien, size_t nouvelle_taille);
```

`realloc` change la taille d'un bloc déjà alloué **en conservant son contenu**. Deux scénarios possibles :



Dans les deux cas, `realloc` renvoie l'adresse **à utiliser désormais** (les nouvelles cases ne sont pas initialisées). En cas d'échec, il renvoie `NULL`… et **l'ancien bloc reste valide**.

Piège classique

`t = realloc(t, nouvelle_taille);` est une erreur classique : si `realloc` échoue, `t` reçoit `NULL`, tu perds la seule adresse de l'ancien bloc et tu ne peux plus le libérer. On passe toujours par un pointeur temporaire :

```c
int *tmp = realloc(t, nouvelle_taille);
if (tmp == NULL) {
    /* échec : t est toujours valide, on peut le libérer proprement */
    free(t);
    return 1;
}
t = tmp;            /* succès : on adopte la nouvelle adresse */
```

```c
#include <stdio.h>
#include <stdlib.h>

void afficher(const int *t, int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", t[i]);
    printf("\n");
}

int main(void)
{
    int n = 3;
    int *t = malloc(n * sizeof *t);
    if (t == NULL)
        return 1;
    for (int i = 0; i < n; i++)
        t[i] = (i + 1) * 10;
    printf("Avant  (%d cases, adresse %p) : ", n, (void *)t);
    afficher(t, n);

    int nouvelle_taille = 6;
    int *tmp = realloc(t, nouvelle_taille * sizeof *t);
    if (tmp == NULL) {
        free(t);
        return 1;
    }
    t = tmp;
    for (int i = n; i < nouvelle_taille; i++)   /* on remplit les nouvelles cases */
        t[i] = (i + 1) * 10;
    n = nouvelle_taille;

    printf("Après  (%d cases, adresse %p) : ", n, (void *)t);
    afficher(t, n);

    free(t);
    return 0;
}
```

Résultat (les adresses changent à chaque exécution)

```
Avant  (3 cases, adresse 0x55cae4c992a0) : 10 20 30
Après  (6 cases, adresse 0x55cae4c992a0) : 10 20 30 40 50 60
```

Les trois premières valeurs ont été conservées ; l'adresse peut être identique ou différente selon la place disponible.

Cas particuliers à connaître : `realloc(NULL, taille)` se comporte comme `malloc(taille)`, ce qui est pratique pour démarrer d'un pointeur `NULL`. Évite en revanche `realloc(p, 0)` dont le comportement varie selon les systèmes : utilise `free(p)`.

30. `free` : rendre la mémoire

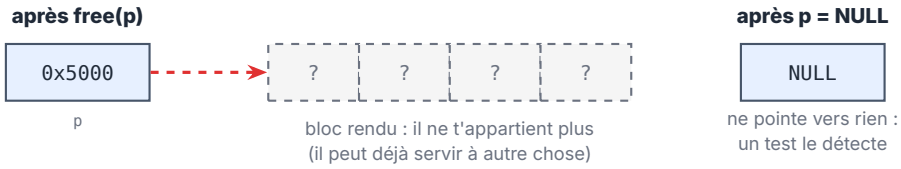
```c
void free(void *p);
```

30.1 Les règles

1. On ne libère que ce qui vient de `malloc`, `calloc` ou `realloc` (jamais l'adresse d'une variable locale ou d'un tableau classique).
2. On libère chaque bloc **une seule fois**.
3. Après `free(p)`, on ne touche plus au bloc : ni lecture, ni écriture.
4. `free(NULL)` ne fait rien (c'est autorisé et sans danger).

30.2 Le pointeur pendant (*dangling pointer*)

`free(p)` rend le bloc au système, **mais ne modifie pas p** : il contient toujours l'ancienne adresse. Il pointe vers une zone qui ne t'appartient plus.



D'où le réflexe : **`free(p); puis p = NULL;`**. Ainsi, une utilisation accidentelle plantera immédiatement (au lieu de corrompre la mémoire en silence), et un second `free(p)` deviendra `free(NULL)`, sans danger.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(sizeof *p);
    if (p == NULL)
        return 1;
    *p = 7;
    printf("*p = %d\n", *p);

    free(p);
    p = NULL;                   /* réflexe de sécurité */

    if (p == NULL)
        printf("p vaut NULL : on sait qu'il ne faut plus l'utiliser\n");

    free(p);                    /* free(NULL) : ne fait rien, aucun risque */
    printf("Fin du programme sans erreur\n");
    return 0;
}
```

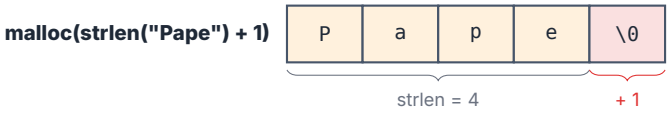
Résultat de l'exécution

```
*p = 7
p vaut NULL : on sait qu'il ne faut plus l'utiliser
Fin du programme sans erreur
```

31. Chaînes de caractères dynamiques

31.1 Le + 1 qu'on n'oublie jamais

Pour copier une chaîne de n caractères, il faut **n + 1** octets : les lettres **et** le `'\0'` final. `strlen` ne compte pas le `'\0'`, d'où le `+ 1`.




```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Renvoie une copie de src dans un bloc neuf (à libérer avec free) */
char *ma_strdup(const char *src)
{
    char *copie = malloc(strlen(src) + 1);     /* + 1 pour le '\0' */
    if (copie == NULL)
        return NULL;
    strcpy(copie, src);
    return copie;
}

int main(void)
{
    char *s = ma_strdup("Bonjour Dakar");
    if (s == NULL)
        return 1;

    s[0] = 'b';                                /* la copie est modifiable */
    printf("Copie : \"%s\" (%zu caractères, %zu octets alloués)\n",
           s, strlen(s), strlen(s) + 1);
    free(s);
    return 0;
}
```

Résultat de l'exécution

```
Copie : "bonjour Dakar" (13 caractères, 14 octets alloués)
```

Astuce

La fonction `strdup` existe déjà sur Linux (et fait partie de la norme C23) : elle fait exactement ce que fait `ma_strdup`. Le résultat doit, lui aussi, être libéré avec `free`.

31.2 Lire un texte puis allouer la taille exacte

On lit dans un grand tampon temporaire, puis on ne garde que la place nécessaire :

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    char tampon[100];
    printf("Ton nom : ");
    if (fgets(tampon, sizeof tampon, stdin) == NULL)
        return 1;
    tampon[strcspn(tampon, "\n")] = '\0';

    char *nom = malloc(strlen(tampon) + 1);
```

```
    if (nom == NULL)
        return 1;
    strcpy(nom, tampon);

    printf("\"%s\" : %zu octets alloués au lieu de %zu\n",
           nom, strlen(nom) + 1, sizeof tampon);
    free(nom);
    return 0;
}
```

Résultat (ce que tu tapes au clavier apparaît aussi)

```
Ton nom : Aminata Ndiaye
"Aminata Ndiaye" : 15 octets alloués au lieu de 100
```

32. Structures dynamiques

32.1 Une structure allouée

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char nom[30];
    int age;
    double moyenne;
} Etudiant;

/* "Constructeur" : crée un étudiant dans le tas (à libérer avec free) */
Etudiant *creer_etudiant(const char *nom, int age, double moyenne)
{
    Etudiant *e = malloc(sizeof *e);     /* sizeof *e = sizeof(Etudiant) */
    if (e == NULL)
        return NULL;
    snprintf(e->nom, sizeof e->nom, "%s", nom);   /* copie sûre du nom */
    e->age = age;
    e->moyenne = moyenne;
    return e;                            /* l'étudiant survit à la fonction */
}

int main(void)
{
    Etudiant *e = creer_etudiant("Awa", 21, 14.5);
    if (e == NULL)
        return 1;

    e->moyenne += 1.5;                   /* pointeur => flèche */
    printf("%s, %d ans, moyenne %.2f\n", e->nom, e->age, e->moyenne);
    printf("Taille d'un Etudiant : %zu octets\n", sizeof *e);

    free(e);
    return 0;
}
```

Résultat de l'exécution

```
Awa, 21 ans, moyenne 16.00
Taille d'un Etudiant : 48 octets
```

32.2 Un tableau dynamique de structures

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char nom[30];
    double moyenne;
} Etudiant;

int main(void)
{
    const char *noms[] = {"Awa", "Moussa", "Sokhna"};
    double moyennes[] = {14.5, 16.0, 12.25};
    int n = 3;

    Etudiant *classe = malloc(n * sizeof *classe);
    if (classe == NULL)
        return 1;

    for (int i = 0; i < n; i++) {
        snprintf(classe[i].nom, sizeof classe[i].nom, "%s", noms[i]);
        classe[i].moyenne = moyennes[i];
    }

    for (int i = 0; i < n; i++)          /* classe[i] est une structure : point */
        printf("%-7s %.2f\n", classe[i].nom, classe[i].moyenne);

    printf("Même chose avec la flèche : %s\n", (classe + 1)->nom);

    free(classe);                        /* un seul malloc => un seul free */
    return 0;
}
```

Résultat de l'exécution

```
Awa     14.50
Moussa  16.00
Sokhna  12.25
Même chose avec la flèche : Moussa
```

À retenir

`classe` est un pointeur, mais `classe[i]` est **une structure** : on écrit donc `classe[i].nom`. Avec un pointeur vers **une** structure (`e`), on écrit `e->nom`. Et `(classe + i)->nom` est équivalent à `classe[i].nom`.

33. Fonctions qui allouent de la mémoire

33.1 Renvoyer un bloc alloué

Contrairement à une variable locale, un bloc du tas **survit** à la fonction qui l'a créé. Une fonction peut donc renvoyer son adresse sans problème. Il faut juste documenter **qui libère** :

```c
#include <stdio.h>
#include <stdlib.h>

/* Crée un tableau de n entiers valant tous `valeur`.
   Renvoie NULL en cas d'échec. L'APPELANT doit libérer avec free(). */
int *creer_tableau(int n, int valeur)
{
    int *t = malloc(n * sizeof *t);
    if (t == NULL)
        return NULL;
    for (int i = 0; i < n; i++)
```

```
        t[i] = valeur;
    return t;
}

int main(void)
{
    int *t = creer_tableau(4, 7);
    if (t == NULL)
        return 1;
    for (int i = 0; i < 4; i++)
        printf("%d ", t[i]);
    printf("\n");
    free(t);                    /* c'est main qui a reçu la clé : il la rend */
    return 0;
}
```

Résultat de l'exécution

```
7 7 7 7
```

33.2 Allouer via un paramètre : le retour du **

Parfois la fonction renvoie déjà un code d'erreur, et doit transmettre le tableau par un paramètre. Piège : si le paramètre est un simple `int *`, la fonction modifie **sa copie** du pointeur.

```c
#include <stdio.h>
#include <stdlib.h>

void allouer_rate(int n, int *t)        /* t est une COPIE du pointeur */
{
    t = malloc(n * sizeof *t);          /* modifie la copie seulement */
}

int main(void)
{
    int *tab = NULL;
    allouer_rate(5, tab);
    printf("tab %s\n", tab == NULL ? "vaut toujours NULL !" : "est alloué");
    return 0;
}
```

Résultat sous Valgrind

```
$ valgrind --leak-check=full ./prog
tab vaut toujours NULL !
==1234== HEAP SUMMARY:
==1234==     in use at exit: 20 bytes in 1 blocks
==1234==   total heap usage: 2 allocs, 1 frees, 4,116 bytes allocated
==1234==
==1234== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1234==    at 0x4846828: malloc (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x10918C: allouer_rate (prog.c:6)
==1234==    by 0x1091B8: main (prog.c:12)
==1234==
==1234== LEAK SUMMARY:
==1234==    definitely lost: 20 bytes in 1 blocks
==1234==    indirectly lost: 0 bytes in 0 blocks
==1234==      possibly lost: 0 bytes in 0 blocks
==1234==    still reachable: 0 bytes in 0 blocks
==1234==         suppressed: 0 bytes in 0 blocks
==1234==
==1234== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Double peine : `tab` vaut toujours `NULL`, **et** le bloc alloué est perdu (Valgrind signale 20 octets « *definitely lost* », alloués dans `allouer_rate`). Et gcc ne dit rien : seul Valgrind repère ce bug. Pour modifier un `int *` de l'appelant, il faut lui passer son adresse, donc un **`int **`** (section 22) :

```c
#include <stdio.h>
#include <stdlib.h>

/* Renvoie 0 si OK, -1 en cas d'échec. Le tableau est transmis via *resultat. */
int allouer_carres(int n, int **resultat)
{
    int *t = malloc(n * sizeof *t);
    if (t == NULL)
        return -1;
    for (int i = 0; i < n; i++)
        t[i] = i * i;
    *resultat = t;                      /* on écrit dans le pointeur de main */
    return 0;
}

int main(void)
{
    int *carres = NULL;
    if (allouer_carres(6, &carres) != 0) {   /* on passe l'ADRESSE du pointeur */
        printf("Échec d'allocation\n");
        return 1;
    }
    for (int i = 0; i < 6; i++)
        printf("%d ", carres[i]);
    printf("\n");
    free(carres);
    return 0;
}
```

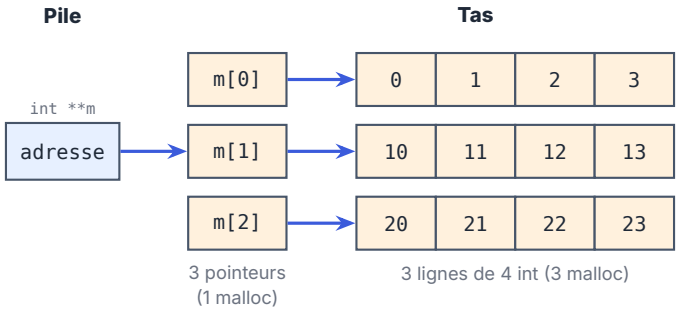
Résultat de l'exécution

```
0 1 4 9 16 25
```

34. Les matrices dynamiques

34.1 Méthode 1 : un tableau de pointeurs vers des lignes (`int **`)

On alloue d'abord un tableau de pointeurs (un par ligne), puis chaque ligne séparément. Avantage : on écrit `m[i][j]` comme avec un tableau 2D classique.




```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int lignes = 3, colonnes = 4;

    int **m = malloc(lignes * sizeof *m);          /* 1. le tableau de pointeurs */
    if (m == NULL)
        return 1;
    for (int i = 0; i < lignes; i++) {
        m[i] = malloc(colonnes * sizeof *m[i]);    /* 2. chaque ligne */
```

```
        if (m[i] == NULL) {                     /* échec : on nettoie ce qui existe */
            for (int k = 0; k < i; k++)
                free(m[k]);
            free(m);
            return 1;
        }
    }

    for (int i = 0; i < lignes; i++)
        for (int j = 0; j < colonnes; j++)
            m[i][j] = i * 10 + j;

    for (int i = 0; i < lignes; i++) {
        for (int j = 0; j < colonnes; j++)
            printf("%4d", m[i][j]);
        printf("\n");
    }

    for (int i = 0; i < lignes; i++)            /* 3. libérer les lignes...   */
        free(m[i]);
    free(m);                                    /* 4. ... PUIS le tableau     */
    return 0;
}
```

Résultat de l'exécution

```
   0   1   2   3
  10  11  12  13
  20  21  22  23
```

Piège classique

L'ordre de libération est l'inverse de l'allocation : d'abord les lignes, **ensuite** `m`. Si tu fais `free(m)` en premier, tu perds les adresses des lignes et tu ne peux plus les libérer.

34.2 Méthode 2 : un seul bloc

On alloue `lignes × colonnes` cases d'un coup et on calcule la position : la case (i, j) est à l'indice `i * colonnes + j`. Un seul `malloc`, un seul `free`, et des données contiguës (plus rapide).

c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int lignes = 3, colonnes = 4;
    int *m = malloc(lignes * colonnes * sizeof *m);
    if (m == NULL)
        return 1;

    for (int i = 0; i < lignes; i++)
        for (int j = 0; j < colonnes; j++)
            m[i * colonnes + j] = i * 10 + j;    /* case (i, j) */

    for (int i = 0; i < lignes; i++) {
        for (int j = 0; j < colonnes; j++)
            printf("%4d", m[i * colonnes + j]);
        printf("\n");
    }
    free(m);
    return 0;
}
```

Résultat de l'exécution

```
 0  1  2  3
10 11 12 13
20 21 22 23
```

	Méthode 1 (`int **`)	Méthode 2 (un bloc)
Accès	`m[i][j]`	`m[i * colonnes + j]`
Nombre de `malloc` / `free`	1 + nombre de lignes	1
Lignes de tailles différentes	possible	non
Libération	lignes puis tableau	un seul `free`

35. Les erreurs classiques... et Valgrind pour les trouver

35.1 Valgrind, ton détecteur de fuites

Les erreurs de mémoire sont sournoises : le programme semble souvent marcher. **Valgrind** exécute ton programme en surveillant chaque accès mémoire et chaque `malloc` / `free`.

```c
gcc -std=c11 -Wall -Wextra -g prog.c -o prog     # -g : pour avoir les numéros de ligne
valgrind --leak-check=full ./prog
```

À la fin, on veut lire `All heap blocks were freed -- no leaks are possible` et `ERROR SUMMARY: 0 errors`. Une alternative intégrée à gcc : compiler avec `-fsanitize=address`, qui arrête le programme dès la première erreur.

35.2 Erreur 1 : la fuite mémoire (oublier `free`)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *t = malloc(10 * sizeof *t);
    if (t == NULL)
        return 1;
    for (int i = 0; i < 10; i++)
        t[i] = i;
    printf("t[9] = %d\n", t[9]);
    return 0;                    /* oubli de free(t) ! */
}
```

Résultat sous Valgrind

```
$ valgrind --leak-check=full ./prog
t[9] = 9
==1234== HEAP SUMMARY:
==1234==     in use at exit: 40 bytes in 1 blocks
==1234==   total heap usage: 2 allocs, 1 frees, 4,136 bytes allocated
==1234==
==1234== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1234==    at 0x4846828: malloc (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x10917E: main (prog.c:6)
==1234==
==1234== LEAK SUMMARY:
==1234==    definitely lost: 40 bytes in 1 blocks
==1234==    indirectly lost: 0 bytes in 0 blocks
==1234==      possibly lost: 0 bytes in 0 blocks
==1234==    still reachable: 0 bytes in 0 blocks
==1234==         suppressed: 0 bytes in 0 blocks
==1234==
```

```
==1234== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Valgrind indique : 40 octets « *definitely lost* », alloués par `malloc` à la ligne 6 de `main`. Dans un petit programme, le système récupère tout à la fin ; mais dans un serveur qui tourne des jours, ou dans une boucle, chaque fuite s'accumule jusqu'à épuiser la mémoire.

Variante sournoise : **écraser la seule adresse d'un bloc**. `p = malloc(100); p = malloc(200);` : le premier bloc n'est plus accessible, il ne pourra jamais être libéré.

35.3 Erreur 2 : ne pas vérifier `NULL`

`malloc` peut échouer. C'est rare sur un PC pour de petits blocs, mais cela arrive (demande énorme, mémoire saturée, systèmes embarqués) :

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    size_t enorme = (size_t)1 << 62;          /* environ 4,6 milliards de Go */
    int *t = malloc(enorme);
    if (t == NULL) {
        printf("malloc a renvoyé NULL : impossible d'allouer %zu octets\n", enorme);
        return 1;
    }
    free(t);
    return 0;
}
```

Résultat de l'exécution

```
malloc a renvoyé NULL : impossible d'allouer 4611686018427387904 octets
```

Sans le test, la ligne suivante qui écrit dans `t` provoquerait un crash.

35.4 Erreur 3 : déborder du bloc

```c
#include <stdlib.h>

int main(void)
{
    int *t = malloc(5 * sizeof *t);
    if (t == NULL)
        return 1;
    for (int i = 0; i <= 5; i++)     /* <= : un tour de trop, écrit t[5] */
        t[i] = i;
    free(t);
    return 0;
}
```

Résultat sous Valgrind

```
$ valgrind --leak-check=full ./prog
==1234== Invalid write of size 4
==1234==    at 0x1091B1: main (prog.c:9)
==1234==  Address 0x4a7e054 is 0 bytes after a block of size 20 alloc'd
==1234==    at 0x4846828: malloc (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x10917E: main (prog.c:5)
==1234==
==1234== HEAP SUMMARY:
==1234==     in use at exit: 0 bytes in 0 blocks
==1234==   total heap usage: 1 allocs, 1 frees, 20 bytes allocated
==1234==
==1234== All heap blocks were freed -- no leaks are possible
==1234==
==1234== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Ici, gcc ne dit **rien** et le programme semble fonctionner normalement. Seul Valgrind repère l'écriture de 4 octets « *0 bytes after a block of size 20* » : juste après la fin du bloc.

35.5 Erreur 4 : utiliser un bloc après `free` (*use after free*)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(sizeof *p);
    if (p == NULL)
        return 1;
    *p = 42;
    free(p);
    printf("%d\n", *p);       /* lecture d'un bloc déjà rendu */
    return 0;
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘main’:
prog.c:11:5: warning: pointer ‘p’ used after ‘free’ [-Wuse-after-free]
   11 |     printf("%d\n", *p);       /* lecture d'un bloc déjà rendu */
      |     ^~~~~~~~~~~~~~~~~
prog.c:10:5: note: call to ‘free’ here
   10 |     free(p);
      |     ^~~~~~
```

Résultat sous Valgrind

```
$ valgrind --leak-check=full ./prog
42
==1234== Invalid read of size 4
==1234==    at 0x1091CB: main (prog.c:11)
==1234==  Address 0x4a7e040 is 0 bytes inside a block of size 4 free'd
==1234==    at 0x484988F: free (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x1091C6: main (prog.c:10)
==1234==  Block was alloc'd at
==1234==    at 0x4846828: malloc (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x10919E: main (prog.c:6)
==1234==
==1234== HEAP SUMMARY:
==1234==     in use at exit: 0 bytes in 0 blocks
==1234==   total heap usage: 2 allocs, 2 frees, 4,100 bytes allocated
==1234==
==1234== All heap blocks were freed -- no leaks are possible
==1234==
==1234== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

gcc le soupçonne, et Valgrind le confirme : lecture « *inside a block of size 4 free'd* ». En sécurité informatique, ce type de bug est une source majeure de failles exploitables.

35.6 Erreur 5 : libérer deux fois (*double free*)

```c
#include <stdlib.h>

int main(void)
{
    int *p = malloc(sizeof *p);
    if (p == NULL)
        return 1;
    free(p);
    free(p);                  /* deuxième free du même bloc */
    return 0;
```

```
}
```

Ce que dit le compilateur (gcc -Wall -Wextra)

```
prog.c: In function ‘main’:
prog.c:9:5: warning: pointer ‘p’ used after ‘free’ [-Wuse-after-free]
    9 |     free(p);                 /* deuxième free du même bloc */
      |     ^~~~~~
prog.c:8:5: note: call to ‘free’ here
    8 |     free(p);
      |     ^~~~~~
```

Résultat de l'exécution

```
free(): double free detected in tcache 2
Aborted (core dumped)
```

La bibliothèque C détecte l'incohérence et arrête brutalement le programme. Le remède est toujours le même : `free(p); p = NULL;` (le second `free` devient alors un `free(NULL)` inoffensif).

35.7 Erreur 6 : oublier le + 1 d'une chaîne

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    const char *src = "Dakar";
    char *copie = malloc(strlen(src));    /* 5 octets : il en faut 6 ! */
    if (copie == NULL)
        return 1;
    strcpy(copie, src);                   /* écrit 6 octets (avec '\0') */
    printf("%s\n", copie);
    free(copie);
    return 0;
}
```

Résultat sous Valgrind

```
$ valgrind --leak-check=full ./prog
Dakar
==1234== Invalid write of size 1
==1234==    at 0x484F39E: strcpy (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x109218: main (prog.c:11)
==1234==  Address 0x4a7e045 is 0 bytes after a block of size 5 alloc'd
==1234==    at 0x4846828: malloc (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x1091F3: main (prog.c:8)
==1234==
==1234== Invalid read of size 1
==1234==    at 0x484F234: strlen (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x48EFCDC: puts (ioputs.c:35)
==1234==    by 0x109224: main (prog.c:12)
==1234==  Address 0x4a7e045 is 0 bytes after a block of size 5 alloc'd
==1234==    at 0x4846828: malloc (in vgpreload_memcheck-amd64-linux.so)
==1234==    by 0x1091F3: main (prog.c:8)
==1234==
==1234== HEAP SUMMARY:
==1234==     in use at exit: 0 bytes in 0 blocks
==1234==   total heap usage: 2 allocs, 2 frees, 4,101 bytes allocated
==1234==
==1234== All heap blocks were freed -- no leaks are possible
==1234==
==1234== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
```

L'affichage paraît correct, mais Valgrind voit l'écriture du `'\0'` un octet trop loin, puis des lectures hors du

bloc.

35.8 Erreur 7 : `sizeof` sur un pointeur

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *t = malloc(10 * sizeof *t);
    if (t == NULL)
        return 1;
    printf("sizeof(t)  = %zu : la taille du POINTEUR, pas du bloc\n", sizeof(t));
    printf("sizeof(*t) = %zu : la taille d'UNE case\n", sizeof(*t));
    /* Il est impossible de retrouver le nombre de cases à partir de t :
       il faut le garder dans une variable (ici 10). */
    free(t);
    return 0;
}
```

Résultat de l'exécution

```
sizeof(t)  = 8 : la taille du POINTEUR, pas du bloc
sizeof(*t) = 4 : la taille d'UNE case
```

35.9 Le récapitulatif

Erreur	Symptôme	Comment l'éviter
Oubli de `free`	fuite mémoire (Valgrind : *definitely lost*)	un `free` par `malloc` ; libérer avant chaque `return`
Adresse écrasée	fuite mémoire	ne jamais réaffecter le seul pointeur d'un bloc non libéré
Pas de test `NULL`	crash si l'allocation échoue	`if (p == NULL)` après chaque allocation
Débordement	corruption silencieuse (*Invalid write*)	indices de `0` à `n - 1` ; garder `n` dans une variable
Use after free	valeurs fausses, faille (*Invalid read*)	`p = NULL;` juste après `free(p);`
Double free	`free(): double free detected` + arrêt	`p = NULL;` juste après `free(p);`
Oubli du `+ 1`	écriture hors du bloc	`malloc(strlen(s) + 1)`
`sizeof(pointeur)`	taille fausse (8)	`n * sizeof *p` et garder `n`
`t = realloc(t, ...)`	fuite si échec	passer par un pointeur temporaire
`free` d'une adresse non allouée	crash	ne libérer que ce qui vient de `malloc` / `calloc` / `realloc`

36. Synthèse : un tableau qui grandit tout seul

Ce mini-projet réunit tout : structure, pointeurs, `malloc`, `realloc` avec pointeur temporaire, `free`. Le principe (utilisé par les `vector` du C++ ou les listes de Python) : on réserve un peu de place, et quand c'est plein, on **double** la capacité.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;          /* le bloc dans le tas           */
    size_t taille;      /* nombre de cases utilisées     */
    size_t capacite;    /* nombre de cases réservées     */
} Vecteur;
```

```c
int vec_init(Vecteur *v)
{
    v->taille = 0;
    v->capacite = 2;
    v->data = malloc(v->capacite * sizeof *v->data);
    return (v->data == NULL) ? -1 : 0;
}

int vec_ajouter(Vecteur *v, int valeur)
{
    if (v->taille == v->capacite) {                /* plein : on double */
        size_t nouvelle = v->capacite * 2;
        int *tmp = realloc(v->data, nouvelle * sizeof *tmp);
        if (tmp == NULL)
            return -1;                             /* v->data reste valide */
        v->data = tmp;
        v->capacite = nouvelle;
        printf("  (capacité portée à %zu)\n", nouvelle);
    }
    v->data[v->taille] = valeur;
    v->taille++;
    return 0;
}

void vec_liberer(Vecteur *v)
{
    free(v->data);
    v->data = NULL;
    v->taille = v->capacite = 0;
}

int main(void)
{
    Vecteur v;
    if (vec_init(&v) != 0)
        return 1;

    for (int i = 1; i <= 10; i++) {
        if (vec_ajouter(&v, i * i) != 0) {
            vec_liberer(&v);
            return 1;
        }
    }

    printf("%zu éléments (capacité %zu) : ", v.taille, v.capacite);
    for (size_t i = 0; i < v.taille; i++)
        printf("%d ", v.data[i]);
    printf("\n");

    vec_liberer(&v);
    return 0;
}
```

Résultat sous Valgrind

```
$ valgrind --leak-check=full ./prog
  (capacité portée à 4)
  (capacité portée à 8)
  (capacité portée à 16)
10 éléments (capacité 16) : 1 4 9 16 25 36 49 64 81 100
==1234== HEAP SUMMARY:
==1234==     in use at exit: 0 bytes in 0 blocks
==1234==   total heap usage: 5 allocs, 5 frees, 4,216 bytes allocated
==1234==
```

```
==1234== All heap blocks were freed -- no leaks are possible
==1234==
==1234== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Valgrind confirme : toutes les allocations ont été libérées, aucune erreur. C'est l'objectif à atteindre pour **chacun** de tes programmes.

À retenir

Check-list avant de rendre un programme qui utilise `malloc` :

- ☐ Chaque `malloc` / `calloc` / `realloc` est suivi d'un test `== NULL`.
- ☐ Les tailles s'écrivent `n * sizeof *p` (et `strlen(s) + 1` pour les chaînes).
- ☐ `realloc` passe par un pointeur temporaire.
- ☐ Chaque bloc est libéré exactement une fois, y compris dans les chemins d'erreur.
- ☐ Après `free(p)`, on écrit `p = NULL;`.
- ☐ Les matrices `int **` sont libérées ligne par ligne, puis le tableau de pointeurs.
- ☐ `valgrind --leak-check=full ./prog` affiche `0 errors` et `no leaks are possible`.

Teste-toi

1. Où sont rangés le pointeur `t` et le bloc qu'il désigne après `int *t = malloc(5 * sizeof *t);` ?
2. Quelle différence entre `malloc(5 * sizeof(int))` et `calloc(5, sizeof(int))` ?
3. Pourquoi `t = realloc(t, 2 * n * sizeof *t);` est-il dangereux ?
4. Combien d'octets faut-il allouer pour copier la chaîne `"Thiès"` ? (Attention, piège : è occupe 2 octets en UTF-8.)
5. Dans quel ordre libère-t-on une matrice allouée avec la méthode `int **` ? Pourquoi ?
6. Après `free(p);`, que contient `p` ? Que faut-il écrire juste après ?
7. Une fonction doit allouer un tableau et le transmettre à l'appelant par un paramètre. Quel doit être le type de ce paramètre ?

PARTIE 4

Bonus et fiche mémo

Les fichiers en deux pages, une fiche récapitulative à garder sous la main, les corrigés des « Teste-toi » et des pistes pour aller plus loin.

37. Les fichiers (l'essentiel)

37.1 Écrire puis relire un fichier texte

Un fichier se manipule à travers un **`FILE *`**… encore un pointeur ! On l'obtient avec `fopen`, on lit ou on écrit, puis on le rend avec `fclose` (exactement comme `malloc` / `free`).

```c
#include <stdio.h>

int main(void)
{
    /* 1. Écriture */
    FILE *f = fopen("notes.txt", "w");      /* "w" : crée ou écrase */
    if (f == NULL) {
        perror("fopen");                    /* affiche la raison de l'échec */
        return 1;
    }
    fprintf(f, "Awa 14.5\nMoussa 16\nSokhna 12.25\n");
    fclose(f);

    /* 2. Lecture */
    f = fopen("notes.txt", "r");            /* "r" : lecture seule */
    if (f == NULL) {
        perror("fopen");
        return 1;
    }
    char nom[30];
    double note, somme = 0;
    int n = 0;
    while (fscanf(f, "%29s %lf", nom, &note) == 2) {   /* 2 valeurs lues ? */
        printf("%-7s %5.2f\n", nom, note);
        somme += note;
        n++;
    }
    fclose(f);

    if (n > 0)
        printf("Moyenne de %d étudiants : %.2f\n", n, somme / n);
    return 0;
```

```
}
```

Résultat de l'exécution

```
Awa     14.50
Moussa  16.00
Sokhna  12.25
Moyenne de 3 étudiants : 14.25
```

`fscanf` renvoie le nombre de valeurs lues : la boucle s'arrête dès qu'une ligne ne contient plus un nom **et** une note (en fin de fichier, il renvoie `EOF`).

37.2 Quand `fopen` échoue

```c
#include <stdio.h>

int main(void)
{
    FILE *f = fopen("fichier_inexistant.txt", "r");
    if (f == NULL) {
        perror("Impossible d'ouvrir fichier_inexistant.txt");
        return 1;
    }
    fclose(f);
    return 0;
}
```

Résultat de l'exécution

```
Impossible d'ouvrir fichier_inexistant.txt: No such file or directory
```

Mode	Effet	Si le fichier n'existe pas
`"r"`	lecture	`fopen` renvoie `NULL`
`"w"`	écriture, **efface** le contenu existant	il est créé
`"a"`	ajout à la fin (*append*)	il est créé
`"r+"`	lecture et écriture	`fopen` renvoie `NULL`
`"rb"`, `"wb"`	versions binaires (images, structures brutes)	comme `"r"` / `"w"`

Pour lire **ligne par ligne**, on utilise `fgets(ligne, sizeof ligne, f)`, qui renvoie `NULL` en fin de fichier :

```c
char ligne[256];
while (fgets(ligne, sizeof ligne, f) != NULL)
    printf("%s", ligne);
```

Piège classique

Toujours tester le retour de `fopen` (fichier absent, droits insuffisants…) et toujours appeler `fclose` : les données écrites restent dans un tampon en mémoire tant que le fichier n'est pas fermé, et peuvent être perdues.

38. Fiche mémo

38.1 Compiler et déboguer

Commande	Usage
`gcc -std=c11 -Wall -Wextra -g prog.c -o prog`	compilation de travail (avertissements + débogage)
`gcc ... prog.c -o prog -lm`	si tu utilises `<math.h>`
`./prog < entree.txt`	exécuter en lisant le clavier depuis un fichier
`valgrind --leak-check=full ./prog`	traquer fuites et accès mémoire invalides
`gcc -fsanitize=address -g prog.c -o prog`	détection des erreurs mémoire intégrée à gcc
`gdb ./prog` puis `run`, `bt`	trouver la ligne d'un *Segmentation fault*

38.2 Formats `printf` / `scanf`

Type	`printf`	`scanf`
`int`	`%d`	`%d` + `&x`
`unsigned int`	`%u`	`%u` + `&x`
`long`	`%ld`	`%ld` + `&x`
`float`	`%f`	`%f` + `&x`
`double`	`%f` (ou `%lf`)	**`%lf`** + `&x`
`char`	`%c`	`" %c"` (avec espace) + `&c`
chaîne	`%s`	`%29s` + `tab` (**sans** `&`)
`size_t`	`%zu`	`%zu` + `&x`
pointeur	`%p` avec `(void *)p`	—

38.3 Pointeurs en un coup d'œil

Écriture	Signification
`int *p;`	déclare un pointeur vers `int`
`int *p = NULL;`	pointeur initialisé à « rien »
`p = &x;`	`p` pointe vers `x`
`*p`	la valeur pointée (lecture ou écriture)
`p = q;`	`p` pointe au même endroit que `q`
`t[i]` ≡ `*(t + i)`	accès à la case `i` (tableau ou pointeur)
`p + 1`	adresse de l'élément suivant (+ `sizeof(*p)` octets)
`q - p`	nombre de cases entre deux pointeurs
`s->champ` ≡ `(*s).champ`	champ d'une structure pointée
`int **pp = &p;`	pointeur de pointeur ; `**pp` = valeur finale
`void f(int *x)` + `f(&n)`	la fonction peut modifier `n`
`void f(int **pp)` + `f(&p)`	la fonction peut modifier le pointeur `p`
`const int *p`	valeur pointée non modifiable via `p`

38.4 Allocation dynamique en un coup d'œil

Fonction	Rôle	Renvoie	À retenir
`malloc(n * sizeof *p)`	réserve `n` cases	adresse ou `NULL`	contenu indéterminé
`calloc(n, sizeof *p)`	réserve `n` cases à zéro	adresse ou `NULL`	tout vaut 0
`realloc(p, n * sizeof *p)`	redimensionne	nouvelle adresse ou `NULL`	passer par un `tmp`
`free(p)`	rend le bloc	rien	une seule fois, puis `p = NULL`
`malloc(strlen(s) + 1)`	place pour copier `s`	adresse ou `NULL`	le `+ 1` du `'\0'`

38.5 Les 10 réflexes anti-bugs

1. Compiler avec `-Wall -Wextra` et corriger **tous** les avertissements.
2. Initialiser toutes les variables (`int somme = 0;`, `int *p = NULL;`).
3. `==` pour comparer, `=` pour affecter ; `strcmp` pour les chaînes.
4. Un `break` à la fin de chaque `case`.
5. Indices de `0` à `n - 1`, jamais `<= n`.
6. Une chaîne de n lettres occupe $n + 1$ cases.
7. `&` devant les variables dans `scanf` (sauf les tableaux de `char`).
8. Passer la taille des tableaux aux fonctions.
9. Après chaque `malloc` / `fopen` : tester `NULL`.
10. Chaque `malloc` a son `free`, chaque `fopen` son `fclose` ; vérifier avec Valgrind.

39. Corrigés des « Teste-toi »

39.1 Partie 1 — Les bases

1. `17 / 5` vaut `3` (division entière), `17 % 5` vaut `2` (le reste), `17 / 5.0` vaut `3.4` (un opérande réel, donc division réelle).
2. Après une saisie, le `'\n'` de la touche Entrée reste en attente. L'espace dans `" %c"` saute tous les blancs (espaces, tabulations, retours à la ligne) avant de lire le caractère ; sans lui, `%c` lirait ce `'\n'`.
3. **6** cases : 5 lettres + le `'\0'` final.
4. `while` teste la condition **avant** chaque tour (il peut ne jamais s'exécuter) ; `do ... while` la teste **après** (au moins un tour).
5. Premier indice : `0`, dernier : `9`.
6. Parce que le C passe les arguments **par valeur** : `x` est une copie. Pour modifier la variable, il faut passer son adresse : `void f(int *x) { *x = 0; }` appelée avec `f(&n);`.

39.2 Partie 2 — Les pointeurs

1. `8` : `p` pointe vers `a`, donc `*p = *p + 3` revient à `a = a + 3`.
2. `*p` vaut `20`, `*(p + 1)` vaut `30`, `p[-1]` vaut `10` (`p` pointe sur `t[1]`).
3. `p` n'est pas initialisé : il contient une adresse au hasard, et `*p = 10` écrit n'importe où (comportement indéfini). Il faut d'abord le faire pointer vers une zone valide : `int x; int *p = &x;` ou `int *p = malloc(sizeof *p);`.
4. `p = &b;` change **la cible** : `p` pointe désormais vers `b`. `*p = b;` copie **la valeur** de `b` dans la variable vers laquelle `p` pointait déjà ; `p` ne bouge pas.
5. `sizeof(t)` vaut `80` (10 `double` de 8 octets), `sizeof(p)` vaut `8` (une adresse).
6. `void minmax(const int *t, int n, int *min, int *max);` (ou `int t[]`), appelée par exemple avec `minmax(t, n, &mini, &maxi);`.
7. Parce qu'un tableau passé en paramètre est converti en pointeur vers sa première case : `sizeof(t)` donne la taille d'une adresse (8), et la fonction n'a aucun moyen de savoir combien de cases suivent. Il faut lui transmettre `n`.

39.3 Partie 3 — L'allocation dynamique

1. Le pointeur `t` (8 octets) est sur la **pile** ; le bloc de 20 octets qu'il désigne est dans le **tas**.
2. `calloc` prend deux arguments (nombre, taille d'un élément) et **met tout à zéro** ; `malloc` prend une taille totale et laisse un contenu **indéterminé**.
3. Si `realloc` échoue, il renvoie `NULL`, qui écrase `t` : l'adresse de l'ancien bloc (toujours valide) est perdue, on ne peut plus ni l'utiliser ni le libérer (fuite). Il faut écrire `int *tmp = realloc(t, ...); if (tmp == NULL) { ... } t = tmp;`.
4. **7 octets** : `T`, `h`, `i`, `s` occupent 1 octet chacun et è en occupe 2 en UTF-8, donc `strlen("Thiès")` vaut 6, plus 1 pour le `'\0'`. `strlen` compte des **octets**, pas des lettres : c'est justement pour ça qu'on écrit `malloc(strlen(s) + 1)` et jamais un nombre deviné.
5. D'abord chaque ligne (`free(m[i])`), **puis** le tableau de pointeurs (`free(m)`). Dans l'autre ordre, on ne pourrait plus lire les `m[i]` après `free(m)` : les lignes seraient perdues.
6. `p` contient **toujours l'ancienne adresse** : c'est un pointeur pendant, qui désigne une zone qui ne t'appartient plus. On écrit `p = NULL;` juste après.
7. Un pointeur de pointeur, par exemple `int **resultat`, et on appelle la fonction avec l'adresse du pointeur : `f(&tab);`.

40. Pour aller plus loin

40.1 La suite logique : les listes chaînées

Avec les pointeurs, les structures et `malloc`, tu as toutes les pièces pour construire des structures de données dynamiques. La plus célèbre : la **liste chaînée**, où chaque élément pointe vers le suivant.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Noeud {
    int valeur;
    struct Noeud *suivant;       /* pointeur vers l'élément suivant */
} Noeud;

/* Ajoute une valeur en tête de liste ; renvoie la nouvelle tête */
Noeud *ajouter_tete(Noeud *tete, int valeur)
{
    Noeud *n = malloc(sizeof *n);
    if (n == NULL)
        return tete;
    n->valeur = valeur;
    n->suivant = tete;           /* le nouveau pointe vers l'ancienne tête */
    return n;
}

int main(void)
{
    Noeud *liste = NULL;                        /* liste vide */
    for (int i = 1; i <= 4; i++)
        liste = ajouter_tete(liste, i * 10);

    for (Noeud *p = liste; p != NULL; p = p->suivant)
        printf("%d -> ", p->valeur);
    printf("NULL\n");

    while (liste != NULL) {                     /* libération élément par élément */
        Noeud *suivant = liste->suivant;        /* on garde la suite AVANT le free */
        free(liste);
        liste = suivant;
    }
    return 0;
}
```

Résultat sous Valgrind

```
$ valgrind --leak-check=full ./prog
40 -> 30 -> 20 -> 10 -> NULL
==1234== HEAP SUMMARY:
==1234==     in use at exit: 0 bytes in 0 blocks
==1234==   total heap usage: 5 allocs, 5 frees, 4,160 bytes allocated
==1234==
==1234== All heap blocks were freed -- no leaks are possible
==1234==
==1234== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

40.2 Pistes de travail

- **Listes chaînées** : insertion en fin, suppression d'un élément, recherche, inversion de la liste.
- **Pointeurs de fonctions** : passer une fonction en paramètre, par exemple à `qsort` de `<stdlib.h>` pour trier n'importe quel tableau.
- **Programmation modulaire** : découper un projet en fichiers `.c` et `.h`, compiler avec un `Makefile`.
- **Débogage** : apprendre les commandes de base de `gdb` (`break`, `next`, `print`, `bt`) et compiler avec `-fsanitize=address,undefined`.
- **Sécurité** : les débordements de tampon, les *use after free* et les chaînes de format mal utilisées sont des familles de failles réelles. Le *SEI CERT C Coding Standard* recense les règles pour les éviter.

40.3 Petits défis pour t'entraîner

1. Écris `void inverser(int *t, int n)` qui inverse un tableau **uniquement avec des pointeurs** (sans `[]`).
2. Écris `char *concatener(const char *a, const char *b)` qui renvoie une nouvelle chaîne allouée contenant `a` suivi de `b`.
3. Lis des entiers au clavier jusqu'à ce que l'utilisateur tape `0`, en stockant tout dans un tableau qui s'agrandit avec `realloc`. Affiche-les à l'envers.
4. Écris les fonctions `creer_matrice`, `remplir`, `afficher` et `liberer_matrice` pour une matrice `int **`, puis vérifie avec Valgrind qu'il n'y a aucune fuite.
5. Ajoute à la liste chaînée une fonction `supprimer(Noeud *tete, int valeur)` qui retire le premier élément ayant cette valeur.

40.4 Références

- *Le langage C* (Kernighan & Ritchie) : la référence historique, écrite par les créateurs du langage.
- *Modern C* (Jens Gustedt) : un livre récent sur le C moderne, disponible gratuitement en ligne.
- **cppreference.com**, section C : la documentation de chaque fonction de la bibliothèque standard.
- Les pages de manuel sous Linux : `man 3 malloc`, `man 3 printf`, `man 3 fopen`…

À retenir

Le C récompense la rigueur : un programme qui compile sans avertissement, qui teste ses `NULL`, qui libère tout ce qu'il alloue et que Valgrind déclare propre, c'est un programme dont tu peux être fier. Bonnes révisions !